

计嵌试卷真题集

2023、2025、2026 年期末试卷

试题册

计算机原理与嵌入式系统

依据原卷逐页校核整理

目录

2022—2023 学年第 2 学期期末试卷	1
2024—2025 学年第 2 学期期末试卷	19
2025—2026 学年第 2 学期期末试卷	36
校核说明	53

本册按年份依次收录三份期末试卷，包含原卷题目、图表和附录。

题文依据原卷逐页校核；原卷自身的文字与代码疑点按原文保留，见卷末校核说明。

2022-2023 学年第 2 学期期末试卷

2022-2023 学年第 2 学期考试试卷

计 算 机 原 理 与 嵌 入 式 系 统

课程编号: EE2003 考试形式: 闭卷

姓名_____ 学号_____ 班级_____

题号	一	二	三	四 ()	四 ()	四 ()	四 ()	总分
得分								

* 第四题中选做的题号请填入上表

第二题 (选择题) 答题卡

6	7	8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23	24	25
26	27	28	29						

第三题 (判断题) 答题卡

30	31	32	33	34	35	36	37	38	39
40	41	42	43						

一、填空题 (10 分, 请直接在试卷上答题)

1. 当代的 CPU 芯片, 往往在单颗处理器芯片内集成冯·诺依曼结构计算机五个基本部件中的_____和_____。
2. 使用流水线后, 指令执行过程会出现不同类型的相关冲突, 其中因程序中分支跳转引起的冲突被称为_____。
此类冲突产生时, “排空” 流水线所带来的延迟被称为_____。
3. I/O 接口的典型电路中, 常包括输入数据 (缓存) 寄存器、输出数据 (缓存) 寄存器、_____、_____等 I/O 端口寄存器。
4. Cortex-M3 处理器进入异常处理子程序时, 被中断程序中下一条待执行指令的地址常被称为_____, 而被中断程序的运行环境 (如 CPU 内部各类寄存器的内容) 常被称为_____。
5. 实验过程中, IDE 采用_____软件 (可进行交叉编译), 宿主机通过_____仿真器 (Debugger) 连接实验箱并烧写代码。

二、单项选择题（36 分，每小题 1.5 分，请在试卷首页答题卡上答题）

6. 以下关于 2023H、0010 1111 0010 0001B、2049D 三个数描述正确的有 ()

 - 若三个数均为无符号数, 则最大的数是 2023H
 - 若三个数均为无符号数, 则最大的数是 0010 1111 0010 0001B
 - 若三个数均为无符号数, 则最大的数是 2049D
 - 若三个数均为无符号数, 则最小的数是 2023H
 - 若三个数均为无符号数, 则最小的数是 0010 1111 0010 0001B
 - 若三个数均为无符号数, 则最小的数是 2049D
 - I、V
 - II、V
 - III、IV
 - III、V
 - IV、V
 - ABCDE 均不正确

7. 某位长为 16 的计算机中, 带符号整数用补码表示。变量 a 、 b 、 c 、 d 的机器数分别为 7FFE_H、0002_H、FFFE_H、8001_H, 下列结论中, 正确的是 ()

 - $a + b$ 运算不会出现溢出
 - $a + c$ 运算不会出现溢出
 - $b + c$ 运算不会出现溢出
 - $b + d$ 运算不会出现溢出
 - $c + d$ 运算不会出现溢出
 - I、II、V
 - I、III、V
 - II、III、IV
 - II、III、V
 - III、IV、V
 - ABCDE 均不正确

8. 已知单精度 float 型数据用 IEEE754 标准 (格式参见附录 5) 表示。假定变量 x 的类型是 float, x 保存在寄存器中, 寄存器值为 F000 0000_H 时, x 的值是 ()

 - -2^{27}
 - 2^{27}
 - -2^{96}
 - 2^{96}
 - -2^{97}
 - 2^{97}

9. 以下关于微程序控制器中机器指令、微指令、微操作描述正确的是 ()

 - I. 每一条机器指令由一段用微指令编成的微程序来解释执行
 - II. 每一条机器指令由一条微指令来执行
 - III. 每一条机器指令的执行可分解为一系列的微操作
 - IV. 每一条微指令由一条机器指令来执行
 - V. 一段机器指令组成的程序可由一条微指令来执行
 - VI. 每一条微指令由若干条机器指令组成

A. I、II B. I、III
C. II、III D. IV、V
E. IV、VI F. ABCDE 均不正确

10. 以下关于计算机体系结构说法正确的有 ()

 - I. 流水线机制能够有效提高 CPI (Cycles Per Instruction)
 - II. 流水线机制能够有效提高 IPC (Instructions Per Cycle)
 - III. Load/Store 体系更有利于实现流水线
 - IV. RISC 相比 CISC 更有利于实现流水线
 - V. 标量处理器可以使用多条流水线来提升性能
 - VI. 超标量处理器可以使用多条流水线来提升性能

A. I、II、III B. II、III、IV
C. III、IV、V D. IV、V、VI
E. III、V、VI F. ABCDE 均不正确

11. 若将流水线分成 Fetch、Decode、Read、Execute、WriteBack 五个阶段，以下描述正确的有 ()

 - I. 不同指令的不同阶段可以重叠执行
 - II. 各个阶段都要共用相同的功能部件
 - III. 各个阶段的执行时间最好不相等
 - IV. 每个阶段的执行时间应取 5 个阶段执行时间的最小值
 - V. 每个阶段的执行时间应取 5 个阶段执行时间的最大值
 - VI. 每个阶段的执行时间应取 5 个阶段执行时间的平均值

A. I、III B. I、IV
C. I、VI D. III、IV
E. IV、VI F. ABCDE 均不正确

12. 以下指令序列（指令功能参考附录 4），会导致流水线数据相关的情形包括（ ）

 - I. 第 1 条指令为“ADD R0, R1, R2”，第 2 条指令为“ADD R3, R4, R5”
 - II. 第 1 条指令为“ADD R0, R1, R2”，第 2 条指令为“ADD R2, R1, R0”
 - III. 第 1 条指令为“LDR R1, [R0]”，第 2 条指令为“ADD R0, R1, R2”
 - IV. 第 1 条指令为“LDR R0, [R1]”，第 2 条指令为“ADD R3, R4, R5”
 - V. 第 1 条指令为“ADD R0, R1, R2”，第 2 条指令为“LDR R1, [R0]”
 - VI. 第 1 条指令为“ADD R3, R4, R5”，第 2 条指令为“LDR R1, [R0]”

A. I、II、III B. II、III、IV
C. II、III、V D. III、V、VI
E. II、IV、VI F. ABCDE 均不正确

13. 在多级存储体系中，缓存——主存结构的作用是解决（ ）问题。

A. 主存与 CPU 速度不匹配 B. 主存和辅存速度不匹配
C. 辅存与 CPU 速度不匹配 D. 外设与 CPU 速度不匹配
E. 主存容量不足 F. 辅存容量不足

14. 下列关于缓存（Cache）地址映射方式说法正确的是（ ）

 - I. 全相联映射方式的缓存命中率一般会高于直接相联映射方式
 - II. 直接相联映射方式的缓存命中率一般会高于组相联映射方式
 - III. 直接相联映射方式的硬件电路比全相联映射方式要简单
 - IV. 组相联映射方式中，Cache 组内的块数一般取值较小（如 16）
 - V. 组相联映射方式中，Cache 的组数目一般取值较小（如 16）
 - VI. 实验所用 STM32F407 系列处理器采用了组相联映射方式
 - VII. PC 上所用处理器一般会采用组相联映射方式

A. I、II B. III、IV
C. IV、V D. V、VI
E. VI、VII F. ABCDE 均不正确

15. 以下关于半导体存储器说法正确的是 ()

- I. 半导体存储与磁存储、光存储相比, 具有体积小、重量轻的优点
- II. 半导体存储被广泛用于芯片内存储、计算机主存和外存
- III. U 盘使用了半导体存储技术
- IV. 硬磁盘 (HDD) 使用了半导体存储技术
- V. 现代计算机上的内存条使用了半导体存储技术
- VI. 实验所用 STM32F407 系列芯片使用了半导体存储技术

- A. I、II、III
- B. I、III、IV
- C. II、III、IV
- D. III、IV、V
- E. IV、V、VI
- F. ABCDE 均不正确

16. 以下关于计算机辅存接口说法正确的是 ()

- I. 早期 PC 往往采用 IDE (Integrated Drive Electronics) 和 ATA (Advanced Technology Attachment) 连接硬磁盘 (HDD)
- II. 当代 PC 往往采用 SATA 连接硬磁盘 (HDD)
- III. 当代 PC 可以采用 SATA 连接固态硬盘 (SSD)
- IV. 当代 MCU 系统往往采用 SATA 连接硬磁盘 (HDD)
- V. 当代 MCU 系统可以采用 SPI 连接外部存储芯片
- VI. 当代 MCU 系统可以采用 I²C 连接外部存储芯片

- A. I、II、III
- B. I、IV、VI
- C. III、IV、V
- D. III、IV、VI
- E. IV、V、VI
- F. ABCDE 均不正确

17. 以下关于计算机总线说法正确的是 ()

- I. 总线的设计思想是把不同部件均接到一个公共的通道上
- II. 总线上的设备一般分为总线主设备和总线从设备
- III. 连接在同一套总线上的所有设备必须按照统一的时钟工作
- IV. 多主设备环境才需要总线仲裁
- V. 单主设备环境也需要总线仲裁
- VI. 单主设备的总线系统中, 总线操作周期可仅包含寻址和数据传输 2 个阶段

- A. I、II、III
- B. I、II、IV
- C. II、III、IV
- D. III、IV、V
- E. IV、V、VI
- F. ABCDE 均不正确

18. 以下关于“地址”描述不正确的是 ()

- I. 地址总线不同信号线对应到地址的不同位
- II. 一般来说, 计算机存储器地址按照字节编码
- III. 地址可以用来寻址存储器内的存储单元
- IV. 地址不能用来寻址 I/O 端口寄存器
- V. 缓存 (Cache) 地址空间和主存地址空间是相同的地址空间
- VI. 虚存空间中的虚拟地址可以转换为物理地址

- A. I、II
- B. II、III
- C. III、IV
- D. IV、V
- E. V、VI
- F. ABCDE 均不正确

19. 以下关于 AHB 总线描述不正确的有 ()

- I. AHB 总线支持多主设备
- II. AHB 总线的地址总线 and 数据总线是复用的
- III. AHB 的读数据和写数据复用同一组信号线
- IV. AHB 总线采用了异步总线时序
- V. AHB 中, “驱动地址”和“驱动数据”可构成两级“流水线”操作
- VI. AHB 总线的读写通道是分离的

- A. I、II
- B. I、III
- C. II、III
- D. III、V
- E. IV、VI
- F. ABCDE 均不正确

20. 以下关于 AHB 突发传输描述不正确的有 ()

- I. 突发传输可以通过 HSIZE[2:0] 指定传输数据块的长度
- II. 突发传输通过 HTRANS[1:0] 指示传输状态如“SEQ”、“IDLE”等
- III. 突发传输在传输开始前必须指定传输数据的长度
- IV. 突发传输所传输数据的地址既可以顺序递增也可以循环递增
- V. 突发传输通过支持数据块的连续传输来提高传输效率
- VI. WRAP4 类型的突发传输支持的传输数据块长度必须是 4 个字

- A. I、II
- B. II、III
- C. III、IV
- D. IV、V
- E. III、VI
- F. ABCDE 均不正确

21. 以下关于 Cortex-M3 中断向量表说法正确的是 ()

 - I. 中断向量表缺省是保存在数据 (SRAM) 区的
 - II. 中断向量表不可以保存在数据 (SRAM) 区
 - III. 中断向量表缺省是保存在代码 (CODE) 区的
 - IV. 中断向量表中保存的是跳转指令
 - V. 中断向量表中保存的是中断服务子程序的入口地址
 - VI. 中断向量表中每个表项占用 4B

A. I、II、III B. II、III、IV
C. II、IV、V D. III、IV、VI
E. III、V、VI F. ABCDE 均不正确

22. 某基于 Cortex-M4 的计算机，存储空间按照字节编址，那么一个对准存放的 unsigned int 类型临时变量（字数据）可能的起始地址是 ()

 - I. 0x0000 0000
 - II. 0x2000 0000
 - III. 0xFF00 0000
 - IV. 0x2022 0004
 - V. 0x2023 0002
 - VI. 0x2023 0008

A. I、II、III B. II、IV、VI
C. III、IV、V D. III、V、VI
E. IV、V、VI F. ABCDE 均不正确

23. 以下关于实验所用 STM32F407 系列处理器描述正确的是 ()

 - I. 复位后具有非特权访问等级
 - II. 复位后使用主栈指针 MSP
 - III. 具有 Thumb 状态和调试状态两种操作状态
 - IV. 具有处理模式和线程模式两种操作模式
 - V. 非特权访问等级可以访问 4GB 空间的各个区域
 - VI. 非特权访问等级可以访问所有的寄存器

A. I、II、III B. I、II、IV
C. II、III、IV D. II、IV、V
E. II、IV、VI F. ABCDE 均不正确

24. 以下关于 Cortex-M3/M4 存储器子系统说法正确的是 ()

- I. 既可以配置为大端模式，也可以配置为小端模式
- II. 外部 RAM 区用于存放数据，不能存放代码
- III. 具有 4GB 线性地址空间
- IV. 内置写缓冲，可以提高程序的执行速度
- V. 地址段 0x0000 0000--0x0000 FFFF 属于代码 (CODE) 区
- VI. 处理器读存储器时，存储器系统会检查所访问区域的存储器访问属性

- A. I、II、III
- B. I、II、IV
- C. II、III、IV
- D. II、IV、V
- E. III、V、VI
- F. ABCDE 均不正确

25. 以下关于位带 (Bit-Band) 操作说法正确的是 ()

- I. 存储空间 CODE 区域中有 1MB 位段区被映射为 32MB 位带别名区域
- II. 存储空间片上外设区域中有 1MB 位段区被映射为 32MB 位带别名区域
- III. 存储空间片外外设区域中有 1MB 位段区被映射为 32MB 位带别名区域
- IV. 存储空间 SRAM 区域中有 1MB 位段区被映射为 32MB 位带别名区域
- V. 存储空间外部 RAM 区域中有 1MB 位段区被映射为 32MB 位带别名区域
- VI. 位带操作可以避免“读 → 修改 → 写”问题

- A. I、II
- B. II、III
- C. III、IV
- D. IV、VI
- E. V、VI
- F. ABCDE 均不正确

26. 以下关于 Cortex-M3/M4 堆栈说法不正确的是 ()

- I. 具有双栈的结构，有 MSP 和 PSP 两个堆栈指针
- II. MSP 和 PSP 的值在初始化程序中进行设置
- III. 非特权线程模式只能使用 PSP 而不能使用 MSP
- IV. 往栈内放置一个新的数据后，栈指针数值会增加
- V. 栈空间只能放置在 4GB 空间的 SRAM 区
- VI. 栈空间可以放置在 4GB 空间的任何一个区

- A. I、II、III
- B. II、III、IV
- C. III、IV、V
- D. IV、V、VI
- E. I、V、VI
- F. ABCDE 均不正确

27. 以下关于 STM32F103 系列处理器时钟描述不正确的有 ()

- I. LSE 和 LSI 时钟源是不能与 PLL 配合工作的
- II. HSE 和 HSI 时钟源由于时钟频率高, 需要从芯片外部输入
- III. SYSCLK 既可以由芯片外部时钟源产生, 也可以由芯片内部时钟源产生
- IV. 各个定时器的时钟均可由系统时钟 SYSCLK 分频得到
- V. 独立看门狗模块的时钟可由系统时钟 SYSCLK 分频得到
- VI. 各个 GPIO 端口的时钟可由系统时钟 SYSCLK 分频得到

- A. I、II
- B. II、III
- C. II、V
- D. IV、V
- E. V、VI
- F. ABCDE 均不正确

28. 以下关于“地址”描述正确的有 ()

- I. “I/O 端口地址”既可以和存储器地址统一规划, 也可以独立规划
- II. STM32 处理器中特定的“I/O 端口地址”代表了特定的寄存器
- III. STM32 处理器中 USART1 模块 SR 寄存器的地址不属于“I/O 端口地址”
- IV. STM32 处理器中 GPIOA 模块 ODR 寄存器的地址属于“I/O 端口地址”
- V. “I²C 总线规范中定义的地址”是 I²C 器件地址
- VI. “I²C 总线规范中定义的地址”是 I²C 器件中 I/O 端口的地址

- A. I、II、III
- B. II、III、IV
- C. II、IV、V
- D. III、IV、V
- E. IV、V、VI
- F. ABCDE 均不正确

29. 以下关于 STM32F103 单片机 USART 接口说法正确的有 ()

- I. USART 模块既支持异步串行通信, 也支持同步串行通信
- II. USART 模块可以触发中断
- III. USART 中断的优先级是固定的
- IV. USART 模块使用前不需要配置引脚复用方式
- V. USART 模块可通过对数据寄存器 DR 的读/写实现收/发
- VI. 所有 USART 模块均挂接在 APB1 总线上

- A. I、II
- B. II、III
- C. III、IV
- D. IV、V
- E. V、VI
- F. ABCDE 均不正确

三、判断题 (14 分, 打“√”或“×”, 请在试卷首页答题卡上答题)

30. ARM 处理器摒弃了冯·诺依曼结构, 均采用哈佛结构设计。()

- 31. 存储器芯片的地址线 and 数据线是不可以复用的。()
- 32. 计算机中缓存地址空间和主存地址空间是相同的地址空间。()
- 33. AHB 和 APB 都具备独立的读写数据通道。()
- 34. Cortex-M3 处理器流水线的 3 级是指取指令、指令译码、指令执行。()
- 35. Cortex-M3 系统中, 子优先级高的异常可以打断子优先级低的异常。()
- 36. Cortex-M3 系统中, MSP 的值不能设置为 0x0000 0000。()
- 37. Cortex-M3 系统中, 异常的优先级寄存器只能由特权访问等级代码访问。()
- 38. 所有 Cortex-M3/M4 的存储器访问指令都支持非对齐的数据传输。()
- 39. 指令周期指的是 CPU 执行一条指令的时间。()
- 40. Cortex-R 处理器利用多处理器锁步 (lock-step) 技术来提高可靠性。()
- 41. STM32F407 系列处理器的每一根外部中断线都各自对应一个不同的中断向量。()
- 42. 微处理器最小硬件系统一般不包括 SPI、I²C 模块。()
- 43. STM32F103 系列 MCU 只有 16 位的定时器, 故看门狗的最大时延是 65536 个 SYSCLK 周期。()

四、设计及综合题（40 分，每题 10 分，6 题选做 4 题，在稿纸上答题）

44.（10 分）

已知图 1 为某 Cortex-M3 系统中总线上数据传输的某个片段。综合运用课程知识，并参考附录 3 信息，回答下列问题。

- （1）图 1 所示时序描述的 AHB 传输中数据的传输方向是什么？
- （2）HREADY 信号起什么作用？
- （3）写出数据 B 和数据 C 对应的地址。
- （4）若数据 B 的值为“0x2000 2000”、数据 C 的值为“0x0800 0800”，分析数据 B、数据 C 各有何含义？
- （5）依据 Cortex-M3 存储空间的区域定义，图 1 中总线操作访问的是哪个存储器区域？并简要说明该区域的主要用途。

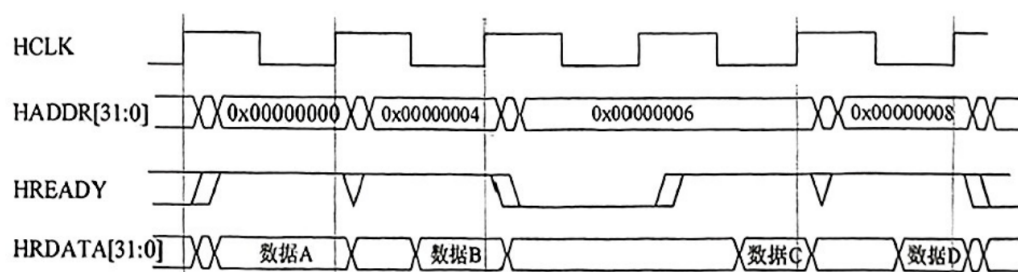


图 1 某 Cortex-M 系统 AHB 总线上的数据传输片段

45.（10 分）

某计算机系统采用 STM32F103ZE 处理器（Cortex-M3，32 根地址线，32 根数据线）。现有数量充足的规格为 1MB × 16 位的 SRAM 芯片，试为该计算机设计一个具有 4MB 容量的片外扩展 SRAM 存储器子系统。（提示：Cortex-M3 定义地址范围 0x6000 0000--0x9FFF FFFF 是外部 RAM 区域。）

- （1）计算需要该型号 SRAM 芯片的数量。
- （2）画出地址线、数据线和 SRAM 芯片 CS 信号线的连接示意图（无需画出每根信号线和完整芯片，能展示信号连接关系即可）。
- （3）列出该 4MB 存储器的地址范围。

46.（10 分）

某计算机主存地址空间大小为 256MB，按字节编址。指令 Cache 和数据 Cache 分离，均有 8 个 Cache 行，每个 Cache 行（cache line）大小为 64B，数据 Cache 采用直接相联映射方式。现有两个功能相同的程序 1 和程序 2，其伪代码如下所示。

程序 1

```
int a[64][64];
int sum_array1() {
    int i, j, sum=0;
    for(i=0; i<64; i++)
        for(j=0; j<64; j++)
            sum += a[i][j];
    return sum;
}
```

程序 2

```
int a[64][64];
int sum_array2() {
    int i, j, sum=0;
    for(j=0; j<64; j++)
        for(i=0; i<64; i++)
            sum += a[i][j];
    return sum;
}
```

假定 `int` 类型数据用 32 位补码表示；程序编译时 `i`, `j`, `sum` 分配在寄存器中；数组 `a` 按行优先方式存放，其首地址为 20D。回答下列问题，要求说明理由或给出计算过程。

- (1) 每个 Cache 行的 Tag 信息需要多少位？
- (2) 若不考虑用于 Cache 一致性维护和替换算法的控制位，则数据 Cache 的总容量（含 Tag 等信息）为多少？
- (3) 数组元素 `a[0][31]` 和 `a[2][1]` 各自所在的主存块对应的 Cache 行号分别是多少？（Cache 行号从 0 开始。）
- (4) 定性说明哪个程序的执行时间更短？
- (5) 计算程序 1 的数据访问命中率（写出表达式即可）。

47. (10 分)

已知实验所用 STM32F407 实验箱上用拨动开关 DIP0 硬件连接如图 2 所示。拟设计的程序功能为：读取拨动开关 DIP0 状态（不接地时 `PE4==1`、接地时 `PE4==0`），通过 USART 串口模块发送所读取的状态值。部分代码如附录 2 所示。请结合课程知识、实验和附录 2 信息，回答以下问题。

- (1) 代码行①所定义的 `GPIOE_BASE` 的值是什么？
- (2) 代码行②所定义的 `GPIOE` 端口输入数据寄存器 `IDR` 的地址是什么？
- (3) 定性说明代码行③的功能。
- (4) 定性说明代码行④的功能。
- (5) 定性说明代码行⑤的功能。
- (6) 代码行⑥从哪个地址获取数据并赋值给变量 `ui_dip0_value`？
- (7) 代码行⑦和代码行⑧分别访问了 USART 模块的状态寄存器 `SR` 的 `TXE` 和 `RXNE` 标志位，解释这 2 个标志位的含义。

48. (10 分)

在实验所用 STM32F407 实验箱上用拨动开关 DIP0/DIP1 控制 LED0/LED1 亮灭。硬件连接如图 2 所示。拟设计的程序功能为：拨动开关 DIP0/DIP1 由不接地（`PE4==1/PE5==1`）拨动至接地（`PE4==0/PE5==0`）时，触发中断来实现 LED0 和 LED1 亮灭切换。部分代码如附录 1 所示。请结合课程知识，分析附录 1 代码，回答以下问题。

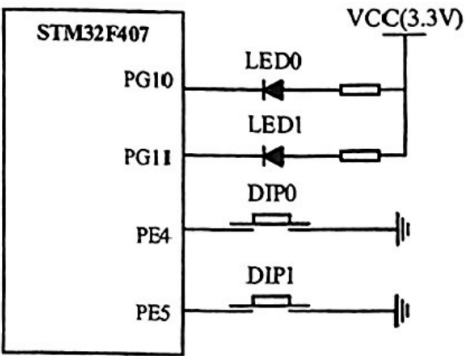


图 2 实验箱的 I/O 引脚连接示意图

- (1) 根据附录 3 表 1 关于 GPIO 寄存器的描述，解释代码行①和②的含义。

(2) 简要说明 `main()` 函数所实现的功能。

(3) 修改 `EXTI9_5_IRQHandler()` 函数代码，实现如下功能：在 `DIP0` 接地（`PE4==0`）时保持原有功能，而 `DIP0` 不接地时点亮 `LED0` 并熄灭 `LED1`。

(4) 如果要修改附录 1 代码，实现如下功能：`DIP0` 拨动至接地（`PE4==0`）时，触发 `EXTI4`；在 `EXTI4` 中断服务子程序中熄灭 `LED0` 和 `LED1`；`EXTI4` 可以打断前述 `EXTI5` 中断服务。定性说明需要对附录 1 代码做哪些修改或补充。

(5) 若希望程序执行更节能，可对 `main()` 函数做哪些修改或补充？定性说明即可。
49. (10 分)
- 图 3 所示为某 MCU 与外设的串行数据传输过程，`SCLK` 为同步时钟，`SID` 为数据。`RW` 为数据传送方向控制：`H` 表示数据从外设到 MCU，`L` 表示数据从 MCU 到外设。`RS` 为数据类型选择：`H` 表示数据是显示内容，`L` 表示数据是控制指令。`D7-D4` 为字节数据的高 4 位，`D3-D0` 为字节数据的低 4 位。传输过程中，只有在时钟线 `SCLK` 拉低时，数据线 `SID` 上的数据才允许变化，在时钟线 `SCLK` 高电平时，`SID` 上的数据必须保持稳定（不能变化）。
-
- 图 3 LCD12864 的串行数据传输过程
- (1) 根据图 3 所示数据传输示意图，发送一个字节数据，实际帧数据有多少字节？

(2) 假设 `SCLK` 和 `SID` 已定义好对应引脚的位带（Bit-Band）操作，即代码“`SCLK=1`”会在 `SCLK` 信号线输出“1”，“`SID=0`”会在 `SID` 信号线输出“0”；可调用的延时函数为 `delay_us(int N)`。编写实现数据串行发送一个字节的函数 `SendByte`。
- 第 13 页（共 53 页）

```
typedef unsigned char u8;
void SendByte(u8 byte)
{
    .....//补充代码
}
```

- (3) 已知该外设某条指令编码为 0x01, 写出 MCU 发该指令操作的代码(调用函数 SendByte 实现)。

附录 1: 中断相关代码 (main.c)

```
1 // 此处略去头文件信息
2 #define LED0_IsOn() GPIO_ReadInputDataBit(GPIOG, GPIO_Pin_10) // 读取 PG10
3 void LED0_Config(void);
4 void KEY0_Config(void);
5 void EXTI_Config(void);
6 void NVIC_Config(void);
7
8 int main(void)
9 {
10     LED0_Config();
11     KEY0_Config();
12     EXTI_Config();
13     NVIC_Config();
14     while (1);
15 }
16
17 void EXTI_Config(void)
18 {
19     EXTI_InitTypeDef EXTI_InitStructure;
20     RCC_APB2PeriphClockCmd(RCC_APB2Periph_SYSCFG, ENABLE);
21     SYSCFG_EXTILineConfig(EXTI_PortSourceGPIOE, GPIO_PinSource5);
22     EXTI_InitStructure.EXTI_Line = EXTI_Line5;
23     EXTI_InitStructure.EXTI_Mode = EXTI_Mode_Interrupt;
24     EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger_Falling;
25     EXTI_InitStructure.EXTI_LineCmd = ENABLE;
26     EXTI_Init(&EXTI_InitStructure);
27 }
28
29 void NVIC_Config(void)
30 {
31     NVIC_InitTypeDef NVIC_InitStructure;
32     NVIC_PriorityGroupConfig(NVIC_PriorityGroup_1);
33     NVIC_InitStructure.NVIC_IRQChannel = EXTI9_5_IRQn;
34     NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0;
35     NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
36     NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
37     NVIC_Init(&NVIC_InitStructure);
38 }
```

```
39
40 void LED0_Config(void)
41 {
42     GPIO_InitTypeDef GPIO_InitStructure;
43     RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOE |
44                             RCC_AHB1Periph_GPIOG, ENABLE);
45     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_11 | GPIO_Pin_10;
46     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT;
47     GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
48     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
49     GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;
50     GPIO_Init(GPIOG, &GPIO_InitStructure);
51 }
52
53 void KEY0_Config(void)
54 {
55     GPIO_InitTypeDef GPIO_InitStructure;
56     RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOE, ENABLE);
57     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_4 | GPIO_Pin_5;
58     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN;
59     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz;
60     GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;
61     GPIO_Init(GPIOE, &GPIO_InitStructure);
62 }
63
64 void EXTI9_5_IRQHandler(void)
65 {
66     if (EXTI_GetITStatus(EXTI_Line5) != RESET)
67     {
68         if (LED0_IsOn())
69             GPIOG->BSRR = 0x0C000000 // ①
70         else
71             GPIOG->BSRR = 0x00000C00; // ②
72         EXTI_ClearITPendingBit(EXTI_Line5);
73     }
74 }
75
76 void EXTI4_IRQHandler(void)
77 {
78 }
```

附录 2: USART 相关代码 (stm32f4xx.h、main.c)

ST 官方库文件 stm32f4xx.h 代码片段

```

1  #define PERIPH_BASE      ((uint32_t)0x40000000)
2  #define AHB1PERIPH_BASE  (PERIPH_BASE + 0x00020000)
3  #define GPIOE_BASE       (AHB1PERIPH_BASE + 0x1000) // ①
4  #define GPIOE            ((GPIO_TypeDef *) GPIOE_BASE)
5
6  typedef struct
7  {
8      __IO uint32_t MODER;    /*Address offset: 0x00 */
9      __IO uint32_t OTYPER;   /*!Address offset: 0x04 */
10     __IO uint32_t OSPEEDR;   /*!Address offset: 0x08 */
11     __IO uint32_t PUPDR;     /*!Address offset: 0x0C */
12     __IO uint32_t IDR;       /*!Address offset: 0x10 */
13     __IO uint32_t ODR;       /*!Address offset: 0x14 */
14     __IO uint16_t BSRRL;     /*!Address offset: 0x18 */
15     __IO uint16_t BSRRH;     /*!Address offset: 0x1A */
16     __IO uint32_t LCKR;      /*!Address offset: 0x1C */
17     __IO uint32_t AFR[2];    /*!Address offset: 0x20-0x24 */
18 } GPIO_TypeDef;

```

main.c

```

1  #include "stm32f10x.h"
2
3  #define BITBAND(addr, bitnum) ((addr & 0xF0000000) + 0x2000000 + ((addr & 0xFFFFF) << 5) +
    (bitnum << 2))
4  #define MEM_ADDR(addr) *((volatile unsigned long *) (addr))
5  #define BIT_ADDR(addr, bitnum) MEM_ADDR(BITBAND(addr, bitnum))
6  #define GPIOE_IDR_Addr (GPIOE_BASE + 16) // ②
7  #define GPIOE_ODR_Addr (GPIOE_BASE + 20)
8  #define PEin(n) BIT_ADDR(GPIOE_IDR_Addr, n) // ③
9  #define PEout(n) BIT_ADDR(GPIOE_ODR_Addr, n)
10
11 void GPIOD_Config(void)
12 {
13     GPIO_InitTypeDef GPIO_InitStructure;
14     RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOD, ENABLE); // ④
15     GPIO_PinAFConfig(GPIOD, GPIO_PinSource5, GPIO_AF_USART2);
16     GPIO_PinAFConfig(GPIOD, GPIO_PinSource6, GPIO_AF_USART2); // ⑤
17     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_5 | GPIO_Pin_6;
18     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;
19     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
20     GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
21     GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;
22     GPIO_Init(GPIOD, &GPIO_InitStructure);
23 }

```

```
24 int main(void)
25 {
26     unsigned int ui_dip0_value = 0;
27     unsigned char uc_send_ch = 0;
28     unsigned char uc_recv_ch = 0;
29     GPIOD_Config();
30     // GPIOE 初始化代码，此处略去
31     // USART2 初始化代码，此处略去
32     while (1) {
33         ui_dip0_value = PEin(4); // ⑥
34         // unsigned int 类型转换为 unsigned char
35         uc_send_ch = ui_dip0_value;
36         if (USART_GetFlagStatus(USART1, USART_FLAG_TXE) == SET) // ⑦
37             USART_SendData(USART2, uc_send_ch); // 向 USART2 发送数据
38         if (USART_GetFlagStatus(USART2, USART_FLAG_RXNE) != RESET) // ⑧
39             uc_recv_ch = USART_ReceiveData(USART2); // 从 USART2 接收数据
40     }
41 }
```

附录 3：STM32F407 系列 MCU 相关信息

表 1 STM32F407 的 GPIO_BSRR 寄存器位定义

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0

GPIO_BSRR[31:16] BRy: Port x reset bit y ($y = 0 \cdots 15$)
BRy 为 1，表示 Port x 的输出数据寄存器（ODRx）的 y 位置 0，即 Port x 的 y 位输出 0。

GPIO_BSRR[15:0] BSy: Port x set bit y ($y = 0 \cdots 15$)
BSy 为 1，表示 Port x 的输出数据寄存器（ODRx）的 y 位置 1，即 Port x 的 y 位输出 1。

表 2 STM32F407 系列 MCU 的中断向量表（部分）

Position	Priority	Type of priority	Acronym	Description	Address
-	-	-	-	Reserved	0x0000 0000
-	-3	fixed	Reset	Reset	0x0000 0004
-	6	settable	SysTick	System tick timer	0x0000 003C
0	7	settable	WWDG	Window Watchdog interrupt	0x0000 0040
10	17	settable	EXTI4	EXTI Line0 interrupt	0x0000 0068
23	30	settable	EXTI9_5	EXTI Line[9:5] interrupts	0x0000 009C
28	35	settable	TIM2	TIM2 global interrupt	0x0000 00B0
37	44	settable	USART1	USART1 global interrupt	0x0000 00D4
38	45	settable	USART2	USART2 global interrupt	0x0000 00D8
39	46	settable	USART3	USART3 global interrupt	0x0000 00DC

附录 4: ARM 汇编指令相关信息

ADD 指令

加法指令，典型语法是“ADD Rd, Rn, Rm”，该指令将 $Rn + Rm$ 的结果存入 Rd。

LDR 指令

从存储器读取字数据至寄存器。典型语法为“LDR Rd, [Rn, #offset]”，该指令从存储器位置 $Rn + offset$ 读取一个字存入寄存器 Rd。若指定操作数寻址方式为前变址寻址（从位置 $Rn + offset$ 读取一个字，更新 Rn 为 $Rn + offset$ ），其典型语法为“LDR Rd, [Rn, #offset]！”。若指定操作数寻址方式为后变址寻址（从位置 Rn 读取一个字，更新 Rn 为 $Rn + offset$ ），其典型语法为“LDR Rd, [Rn], #offset”。

附录 5: 数的表示方法

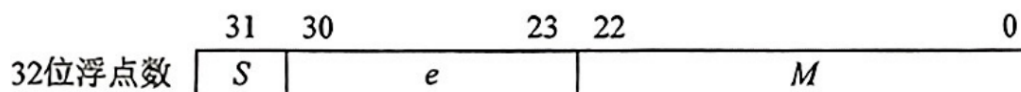
补码

数值 x 的补码记为 $[x]_{\text{补}}$ ，假设机器字长为 n 位，那么数值 x 的补码定义如下：

$$[x]_{\text{补}} = \begin{cases} x, & 0 \leq x \leq 2^{n-1} - 1, \\ 2^n - |x|, & -2^{n-1} \leq x \leq 0. \end{cases}$$

浮点数

在国际电气和电子工程师协会 IEEE-754 标准中，32 位浮点数的格式如下：



S 位是 1bit 符号位， e 是 8bit 阶码（隐含基数为 2）， M 是 23bit 的尾数。IEEE-754 规定，上述格式所表示浮点数的真值为

$$(-1)^S \times 2^{e-127} \times (1 + M).$$

2024-2025 学年第 2 学期期末试卷

中国科学技术大学信息科学技术学院
2024-2025 学年第 2 学期考试试卷

计 算 机 原 理 与 嵌 入 式 系 统

课程编号：EE2003 考试形式：半开卷

姓名_____ 学号_____ 班级_____

■ A 卷 □ B 卷

题号	一	二	三	四（ ）	四（ ）	四（ ）	四（ ）	总分
得分								

* 第四题中选做的题号请填入上表

第二题（选择题）答题卡

11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30

第三题（判断题）答题卡

31	32	33	34	35	36	37	38	39	40
41	42	43	44	45					

一、填空题（15 分，每小题 1 分，直接在试卷上答题）

（一）数据的表示和存取

本课程中实验箱系统采用 STM32F407 芯片，考虑该系统上运行的 C 程序。程序中某变量保存在寄存器 R1，R1 内容为 0xFFFF FFEF。综合运用课程知识，回答如下问题。

1. 若该变量是 32 位无符号整数，变量的真值为_____；若该变量经饱和（Saturation）操作得到 8 位无符号整数，该 8 位无符号数的十进制真值为_____。
注：饱和运算，即运算结果大于（数表示法所能表示的）上限或小于下限时，结果就等于上限或下限。
2. 若该变量是 32 位有符号整数，变量的十进制真值为_____。
3. 若 R1 中数值是一个 8 位有符号整数经符号扩展（Sign Extension）转换至 32 位得到，则该 8 位有符号整数的 16 进制机器数为_____，该 8 位有符号整数的十进制真值为_____。
注：有符号整数进行符号扩展时，扩展的位（即新增的位）将复制原始数值的符号位。

(二) Cortex-M4 内核与存储器之间的数据传输

已知某 STM32F407 系统片内仅实现了 64kB 的 SRAM (起始地址 0x2000 0000)，以及 96kB 的 Flash (起始地址 0x0800 0000)，且无外接存储器。系统支持位段操作，即地址 0x2000 0000 起始的 1MB 会被映射至地址 0x2200 0000 起始的别名区域。图 1 为该系统总线上传输的某个片段；表 1 为某程序执行过程中图 1 操作开始前存储器内存储的部分信息。综合运用课程知识，并参考“附录 5：ARM 汇编指令相关信息”，回答下列问题（提示：数值用 16 进制表示）。

4. 该系统片上 SRAM 可有效寻址的地址范围是_____。
5. 该系统片上 Flash 可有效寻址的地址范围是_____。
6. 上电复位后执行的第一条指令的地址是_____。
7. 若启动代码中设置主栈大小为 256B：
该栈在 4GB 存储空间所占用的地址范围是_____；
第一次压栈的数据值是_____。
8. 参考附录 5 中 LDR 指令格式，若数据 B 相关的总线操作由指令“LDR R1, [R0]”引起，则该指令执行前 R0 保存的数值是_____；该指令执行后 R1 保存的数值是_____。
9. 若数据 D 的传输由指令“LDR R1, [R0]”触发：
该指令执行前 R0 中保存的数值是_____；
该指令执行后 R1 中保存的数值是_____。
10. 若将地址 0x2000 0404 起始的字数据保存到 R4 中，再执行指令“LDR R3, [R4]”，会触发_____。

表 1 AHB 总线传输发生前 STM32F407 片内存储器信息

地址	字数据
0x0000 0000	0x2000 0400
0x0000 0004	0x0800 1000
0x0000 0008	0x0800 2000
...	
0x2000 0000	0xFF00 2025
0x2000 0004	0xFFFF 2026
0x2000 0008	0x00FF 2027
...	
0x2000 0400	0x1234 2023
0x2000 0404	0x2024 FFFF
0x2000 0408	0x00FF 2025

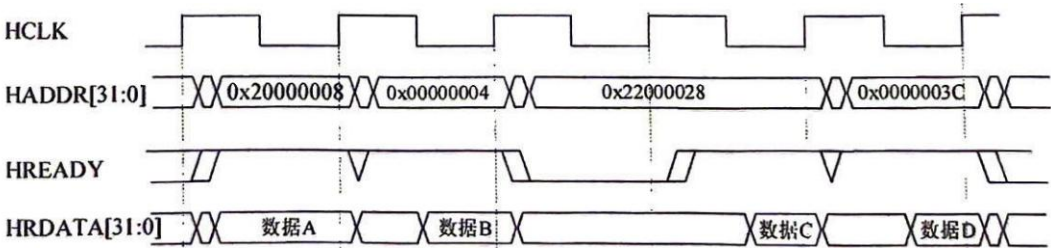


图 1 某 Cortex-M 系统 AHB 总线上的数据传输片段

二、单项选择题（30 分，每小题 1.5 分，在试卷首页答题卡答题）

11. 依据 IEEE-754 标准，4730 0000H 对应的真值是多少（ ）
A. 0.375×2^{14} B. 0.375×2^{15} C. 1.375×2^{14} D. 1.375×2^{15}
12. 某科学实验中需要记录大量的整型参数，为了保证数据精度的基础上提高运算速度，需要选择合理的数据表示方法。若参数 α 和 β 的取值范围分别为 $-2^{20} \sim 2^{20}$ 、 $-2^{40} \sim 2^{40}$ ，则下列选项中， α 和 β 最适宜采用的数据表示方法分别是（ ）
A. 32 位整数，32 位整数 B. 单精度浮点数，单精度浮点数
C. 32 位整数，双精度浮点数 D. 单精度浮点数，双精度浮点数
13. 以下关于 2025H、0010 0000 0010 0100B、2025D 三个数描述正确的有（ ）
I. 若三个数均为无符号数，则最大的数是 2025H
II. 若三个数均为无符号数，则最大的数是 0010 0000 0010 0100B
III. 若三个数均为无符号数，则最大的数是 2025D
IV. 若三个数均为无符号数，则最小的数是 2025H
V. 若三个数均为无符号数，则最小的数是 0010 0000 0010 0100B
VI. 若三个数均为无符号数，则最小的数是 2025D
A. I、V B. II、VI C. I、VI D. II、V
E. III、VI F. A-E 均不合题意
14. 某 Cortex-M 系统中，带符号整数用补码表示。变量 a、b、c、d 的机器数分别为 0x7FFFFFF5、0x0000000F、0xFFFFFFFF、0x80000001，下列哪些运算会导致程序状态寄存器 PSR 的溢出位 $V = 1$ （ ）
A. a-b 运算 B. a+c 运算 C. b+c 运算 D. b+d 运算
E. c+d 运算 F. A-E 均不合题意
15. 在多级存储体系中，缓存——主存结构的作用是解决（ ）问题。
A. 主存容量不足 B. 主存与 CPU 速度不匹配
C. 主存和辅存速度不匹配 D. 辅存与 CPU 速度不匹配
E. 主存和辅存容量不匹配 F. A-E 均不合题意

16. 以下关于流水线相关冲突说法不正确的有（ ）

- I. 数据相关指一条指令需要使用前一条（尚未完成计算）指令的计算结果
 - II. 多个指令同时需要使用同一个寄存器，这属于数据相关冲突
 - III. 程序中的转移指令有可能引起控制相关
 - IV. 数据相关冲突可以通过插入“气泡”的方式消除
 - V. 资源相关冲突可以通过插入“气泡”的方式消除
 - VI. 控制相关冲突可以通过插入“气泡”的方式消除
 - VII. 动态转移预测算法常用来解决控制相关冲突
- A. I、IV B. II、IV C. II、VI D. III、IV
- E. IV、VI F. A-E 均不合题意

17. 以下关于 Cortex-M3/M4 流水线说法正确的有（ ）

- I. 流水线机制能够有效提高 CPI（Cycles Per Instruction）
 - II. 流水线机制能够有效提高 IPC（Instructions Per Cycle）
 - III. 流水线分成取指令、指令解码、指令执行三个阶段
 - IV. 流水线分成取指令、指令解码、取操作数、指令执行、回写结果五个阶段
 - V. 流水线中每个阶段的执行时间应取多个子阶段执行时间的最大值
 - VI. 流水线中每个阶段的执行时间应取多个子阶段执行时间的最小值
 - VII. 流水线中每个阶段的执行时间应取多个子阶段执行时间的平均值
- A. I、III、V B. I、III、VI C. II、III、VI D. II、III、V
- E. II、IV、V F. A-E 均不合题意

18. 以下关于 Cortex-M 系统中指令执行过程描述不正确的是（ ）

- I. PC 寄存器中保存了指令的地址
 - II. 指令可以保存在 4GB 存储空间的代码（CODE）区
 - III. 指令可以保存在 4GB 存储空间的 SRAM 区
 - IV. 取指令操作会用到数据总线，但不会使用地址总线
 - V. 指令可能是 16 比特的，也可能是 32 比特的
 - VI. 指令执行过程只会用到数据总线，但不会使用地址总线
- A. I、II B. II、III C. III、V D. IV、VI
- E. V、VI F. A-E 均不合题意

19. 以下关于缓存（Cache）地址映射方式说法正确的是（ ）
- I. 全相联映射方式的缓存命中率一般会低于直接相联映射方式
 - II. 直接相联映射方式的缓存命中率一般会低于组相联映射方式
 - III. 组相联映射方式的硬件电路比直接相联映射方式要简单
 - IV. 组相联映射方式中，Cache 组内的块数一般取值较小（如 16）
 - V. 组相联映射方式中，Cache 的组数目一般取值较小（如 16）
 - VI. 实验所用 STM32F407 系列处理器采用了直接相联映射方式
 - VII. PC 上的通用 CPU 一般会采用直接相联映射方式
- A. I、II B. I、IV C. II、IV D. II、V
- E. II、VII F. A-E 均不合题意
20. 以下关于半导体存储器说法正确的有（ ）
- I. MCU 片上 Flash 使用了半导体存储技术
 - II. 固态硬盘（SSD）使用了半导体存储技术
 - III. U 盘使用了半导体存储技术
 - IV. 硬磁盘（HDD）使用了半导体存储技术
 - V. PC 的内存条使用了半导体存储技术
 - VI. 半导体存储比磁存储具有更高的可靠性和耐用性
- A. I、II、III B. I、III、IV C. II、III、IV D. III、IV、V
- E. III、IV、VI F. A-E 均不合题意
21. 以下关于“地址”描述不正确的是（ ）
- I. 地址总线中信号线数目影响了可寻址空间的大小
 - II. 存储器地址可以用来标识存储器内的存储单元
 - III. I/O 端口地址和存储器地址必须使用相同的地址线
 - IV. I/O 端口地址和存储器地址必须使用独立的地址线
 - V. 缓存（Cache）地址空间和主存地址空间是相同的地址空间
 - VI. 紧耦合内存（TCM）地址空间和主存地址空间是相同的地址空间
 - VII. 虚存空间中的虚拟地址可以转换为物理地址
- A. I、III、IV B. II、III、V C. III、IV、VI D. III、IV、V
- E. III、V、VI F. A-E 均不合题意

22. 以下关于计算机总线说法不正确的是 ()

- I. 总线的设计思想是把不同部件均接到一个公共的通道上
- II. 同步总线中, 挂接在同一总线上的设备必须按照统一的时钟工作
- III. 异步总线时序可分成不互锁方式、半互锁方式和全互锁方式
- IV. 半同步总线和异步总线中, 收发双方没有公共的时钟
- V. 总线上的设备一般分为总线主设备和总线从设备
- VI. 单主设备的总线中, 总线操作周期可分解为寻址子周期和数据传送子周期
- VII. 无论是单主设备环境还是多主设备环境均需要总线仲裁

- A. II、IV B. II、VII C. IV、VII D. V、VII
E. VI、VII F. A-E 均不合题意

23. 以下关于 AHB 总线描述不正确的有 ()

- I. AHB 总线支持多主设备
- II. AHB 的读数据和写数据复用同一组信号线
- III. AHB 总线采用了同步总线时序
- IV. AHB 总线中插入等待周期的数据传输, 本质上是一种半同步时序
- V. AHB 中, “驱动地址” 和 “驱动数据” 可构成两级 “流水线” 操作
- VI. INCR4 类型的突发传输支持的传输数据块长度必须是 4 个字节
- VII. INCR4 类型的突发传输支持的传输数据块长度必须是 4 个字

- A. II、III、IV B. III、IV、V C. II、IV、VI D. II、IV、VII
E. II、VI、VII F. A-E 均不合题意

24. 某使能了位段操作的 Cortex-M4 系统, 存储空间按照字节编址, 那么一个对准存放的 float 类型临时变量 (字数据) 的起始地址不会是 ()

- I. 0x0000 0000
- II. 0x2025 2024
- III. 0xFF00 2024
- IV. 0x2200 2025
- V. 0x2025 0004
- VI. 0x2025 0008

- A. I、II、III B. I、III、IV C. II、III、IV D. II、V、VI
E. III、IV、V F. A-E 均不合题意

25. 以下关于实验所用 STM32F407 系列处理器描述不正确的是 ()

- I. 复位后默认是非特权访问等级
- II. 复位后使用主栈指针 MSP
- III. 支持 AArch64 和 AArch32 两种运行状态
- IV. 具有处理模式和线程模式两种操作模式
- V. 非特权访问等级可以访问 4GB 空间的所有区域
- VI. 非特权访问等级不能访问所有的寄存器

- A. I、II、III B. II、III、IV C. I、III、V D. II、IV、V
E. IV、V、VI F. A-E 均不合题意

26. 以下关于 Cortex-M3/M4 存储器子系统说法不正确的是 ()

- I. 既可以配置为大端模式，也可以配置为小端模式
- II. 地址段 0x2000 0000--0x0001 FFFF 属于 SRAM 区
- III. 地址段 0x2000 0000--0x0001 FFFF 属于片外外设区
- IV. SRAM 区用于存放数据，不能存放代码
- V. 外部 RAM 区用于存放数据，也可以存放代码
- VI. SRAM 区有 1MB 位段区被映射为 16MB 位带别名区域
- VII. 处理器读存储器时，存储器系统会检查所访问区域的存储器访问属性

- A. I、IV、V B. II、III、IV C. III、IV、V D. III、IV、VI
E. III、V、VII F. A-E 均不合题意

27. 以下关于 Cortex-M3/M4 堆栈说法不正确的是 ()

- I. 具有双栈的结构，有 MSP 和 PSP 两个堆栈指针
- II. 栈的访问一定是字对齐的
- III. 栈的访问需要特权访问等级
- IV. 非特权线程模式只能使用 PSP 而不能使用 MSP
- V. 往栈内放置一个新的数据后，栈指针数值会增大
- VI. 往栈内放置一个新的数据后，栈指针数值会减小
- VII. 栈空间只能配置在 4GB 空间的 SRAM 区

- A. II、III、V B. II、III、VI C. III、V、VII D. III、VI、VII
E. IV、VI、VII F. A-E 均不合题意

28. 以下关于 STM32F103 系列处理器时钟描述不正确的有 ()

- I. SYSCLK 既可以由芯片外部时钟源产生, 也可以由芯片内部时钟源产生
- II. SYSCLK 可以来源于 HSE、LSE、HSI 或 LSI
- III. 各定时器模块的时钟可由系统时钟 SYSCLK 分频得到
- IV. 系统定时器 SysTick 的时钟由系统时钟 SYSCLK 分频得到
- V. 独立看门狗模块的时钟由系统时钟 SYSCLK 分频得到
- VI. 各个 GPIO 端口的时钟由系统时钟 SYSCLK 分频得到

- A. I、II B. I、III C. I、V D. II、IV
E. II、V F. A-E 均不合题意

29. 以下关于 STM32 处理器中 “I/O 端口地址” 描述正确的有 ()

- I. “I/O 端口地址” 是和存储器地址统一规划的
- II. 定时器模块中的控制寄存器 CR1、CR2 的地址不属于 “I/O 端口地址”
- III. USART1 模块中的状态寄存器 SR 的地址不属于 “I/O 端口地址”
- IV. GPIOA 模块中输出寄存器 ODR 的地址属于 “I/O 端口地址”
- V. SPI 模块 CR1、CR2 寄存器的地址不属于 “I/O 端口地址”
- VI. “I²C 总线规范中定义的地址” 不属于 “I/O 端口地址”
- VII. “I²C 总线规范中定义的地址” 是 I²C 器件内部的存储单元地址

- A. I、IV、VI B. I、IV、V C. II、III、V D. II、III、V、VII
E. II、V、VII F. A-E 均不合题意

30. 以下关于 Cortex-M3/M4 中断 (异常) 子系统说法不正确的有 ()

- I. 一般情况下, 中断向量表设计为保存中断服务子程序的入口地址
- II. 中断向量表中每个表项都占据 4 字节
- III. 中断向量表中不同的行存放的字数据不可能是相同的
- IV. 可通过某些特殊的寄存器屏蔽除复位、NMI 和硬件错误以外的所有中断
- V. 子优先级高的异常可以打断组优先级低的异常
- VI. 组优先级高的异常可以打断组优先级低的异常
- VII. 上电复位后, 取出地址为 0x00000000 的第一个字, 作为复位中断向量

- A. I、II、III B. II、III、V C. III、IV、V D. III、IV、VI
E. III、V、VII F. A-E 均不合题意

三、判断题 (15 分, 每小题 1 分, 打 ✓ 或 ×, 在试卷首页答题卡答题)

31. 每一条机器指令的执行可分解为一系列的微操作。()

32. DDR 内存采用了 2bit 的数据预取 (2n-Prefetch)。()

33. EPROM、EEPROM、Flash 都是电可擦除的 ROM。()
34. 现代 PC 上 SATA、PCIe 5.0 和 USB4 都采用了串行接口技术。()
35. AHB 总线主机读取从机数据时需要采样 HWDATA 总线。()
36. 直接存储器访问 (DMA) 在传送大数据块时具有明显优势。()
37. Cortex-M 处理器的写缓冲并不会调整程序中指令执行的顺序。()
38. Cortex-M3 系统中, MSP 的值不能设置为 0x4000 2025。()
39. Cortex-M 处理器通过 MMU 来管理 Cache。()
40. AHB 中突发传输方式控制复杂, 在提高灵活性的同时牺牲了传输性能。()
41. Cortex-M 内核访问 4GB 存储空间的 SRAM 区和 CODE 区时采用了物理上独立的总线。()
42. Cortex-M3 系统中, 访问地址为 0x40001800 的端口寄存器某比特可以使用位带操作避免“读 → 修改 → 写”过程。()
43. STM32F103 系列 MCU 中, 中断向量表有 256 个表项。()
44. AAPCS 规范约定了函数调用时寄存器使用、数据栈使用以及参数传递的规则。()
45. 实验中, 代码“GPIOA->ODR = 0xFFFF0000”可以实现向 GPIOA 端口的 16 个引脚输出高电平 (逻辑“1”)。()

四、设计及综合题 (40 分, 每题 10 分, 5 题选做 4 题, 在稿纸上答题)

(一) CPU 连接存储器

46. (10 分)

某 32 位计算机系统, 地址空间大小为 4GB, 存储器按字节编址。现有数量充足的 SRAM 芯片, 容量为 2-Mbit (规格为 256K × 8-bit)。试为该系统设计一个具有 1MB 容量的片外扩展 SRAM 存储器子系统, 要求该 1MB 处于连续地址范围, 且起始地址是 0x6000 0000。

- (1) [2 分] 计算需要几片 SRAM。
- (2) [6 分] 画出地址线、数据线及片选信号的连接示意图。
- (3) [2 分] 写出上述设计中扩展 SRAM 存储子系统所占用的地址范围。

(二) Cache 及其应用

47. (10 分)

某计算机系统主存地址空间大小为 1MB, 按字节编址。指令 Cache 和数据 Cache 分离, 容量均为 16kB, 均采用 4 路组相联映射方式, 每个 Cache 行 (cache line) 大小为 128B。现有两个功能相同的程序 1 和程序 2, 其伪代码如下所示。

程序 1	程序 2
<pre>int a[64][64]; int sum_array1() { int i, j, sum=0; for(i=0; i<64; i++) for(j=0; j<64; j++) sum += a[i][j]; return sum; }</pre>	<pre>int a[64][64]; int sum_array1() { int i, j, sum=0; for(j=0; j<64; j++) for(i=0; i<64; i++) sum += a[i][j]; return sum; }</pre>

假定 `int` 类型数据用 32 位补码表示，程序编译时 `i`、`j`、`sum` 均分配在寄存器中，数组 `a` 按行优先方式存放，其首地址为 `0x0`，代码运行前 Cache 为空。请回答下列问题，要求说明理由或给出计算过程。

- (1) [2 分] 数据 Cache 被分为多少组？每组内多少个 Cache 行？
- (2) [2 分] 主存被分为多少页？每页内多少个 Cache 行？
- (3) [2 分] 数组 `a` 能否全部载入 Cache？
- (4) [2 分] 数组元素 `a[0][31]` 载入 Cache 时，写出其对应的 Cache 地址范围。
- (5) [2 分] 计算程序 1 和程序 2 的数据访问命中率（写出表达式即可）。

(三) GPIO 及定时器

48. (10 分)

考虑一个 STM32F103 系统，其 GPIO 端口的引脚连接了 LED 灯 (LED0) 和拨码开关 (DIP0)，如图 2 所示。在该系统上执行“附录 1：定时器控制 LED 亮灭相关代码”所示代码。已知该系统配置为使用外部输入的 8MHz 时钟源且未使能 PLL，AHB 预分频系数、APB1 预分频系数、TIM2 倍频系数均设置为 1。参考“附录 4：STM32F103 系列 MCU 相关信息”，回答下列问题。

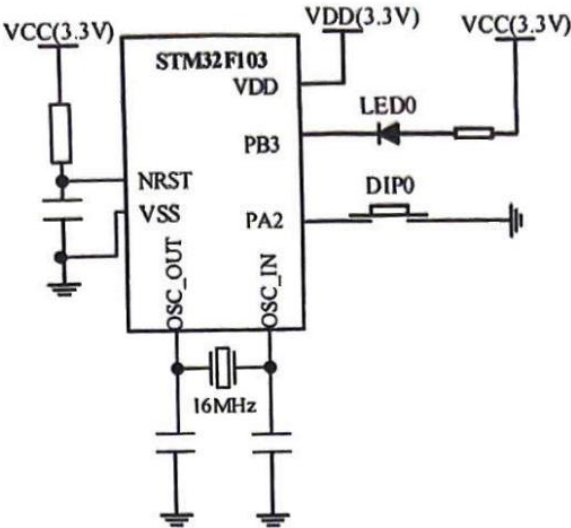


图 2 STM32F103 系统硬件连接示意图

- (1) [2 分] 分析 `TIM2_Delay()` 函数的功能。
- (2) [2 分] 分析 `main()` 函数的功能。

- (3) [2 分] 根据“stm32f10x.h”和“stm32f10x_gpio.c”，分析①行代码所访问的存储单元的起始地址（即 4GB 存储空间中的地址），地址请用 16 进制表示。
- (4) [2 分] 分析②行代码所访问的存储单元的地址范围（即 4GB 存储空间中的地址范围），地址请用 16 进制表示。
- (5) [2 分] 分析③行代码的功能。

（四）GPIO 及中断

49. (10 分)

考虑如图 2 所示系统。在该系统上执行“附录 2：中断控制 LED 的相关代码”所示代码，能够实现拨动开关 DIP0 由不接地（引脚对应位 = 1）拨至接地（引脚对应位 = 0）时，触发外部中断，在中断服务子程序中控制 LED 灯的亮灭。回答下列问题。

- (1) [2 分] 根据“附录 2：中断控制 LED 的相关代码”中关于 GPIO 寄存器的描述，解释代码行①和②的含义。
- (2) [2 分] 补全函数 NVIC_Config() 中③和④行的代码。
- (3) [2 分] 参考“附录 4：STM32F103 系列 MCU 相关信息”，若主程序中只使用了 DIP0 对应的中断（其他中断未配置），那么在函数 EXTI_Config() 执行后，中断屏蔽寄存器 EXTI_IMR 的值是多少？
- (4) [2 分] 简述中断服务子程序的功能。
- (5) [2 分] 简述代码运行后所实现的整体效果（LED 灯亮灭变化规律）。

（五）串行接口通信

50. (10 分)

某基于 STM32F407 的嵌入式系统通过串口转 USB 芯片连接 PC。PC 上运行程序的功能为：若串口接收到的数值为偶数，则通过该串口发送 0xFF；若串口接收到的数值为奇数，则通过该串口发送 0xA5。嵌入式系统上指示灯 LED0、LED1，拨动开关 DIP0、DIP1 的硬件连接如图 3 所示，嵌入式系统运行附录 3 所示代码。请结合课程知识、实验和附录 3，回答以下问题。

- (1) [1 分] 解释代码行①中 USART_FLAG_TC 标志位的含义。
- (2) [2 分] 说明代码行②的功能。
- (3) [1 分] 解释代码行③中 USART_FLAG_TXE 标志位的含义。
- (4) [1 分] 解释代码行④中 USART_FLAG_RXNE 标志位的含义。
- (5) [1 分] 该串口能实现的最高数据传输速率为多少？
- (6) [2 分] 若拨动开关 DIP0、DIP1 均接地，PC 上串口将接收到的数据是什么？
- (7) [2 分] 补全代码行⑤（一行或多行均可），使程序的功能为：接收到 0xFF，则控制 LED0 亮且 LED1 灭；接收到其他数值，则控制 LED0 灭且 LED1 亮。

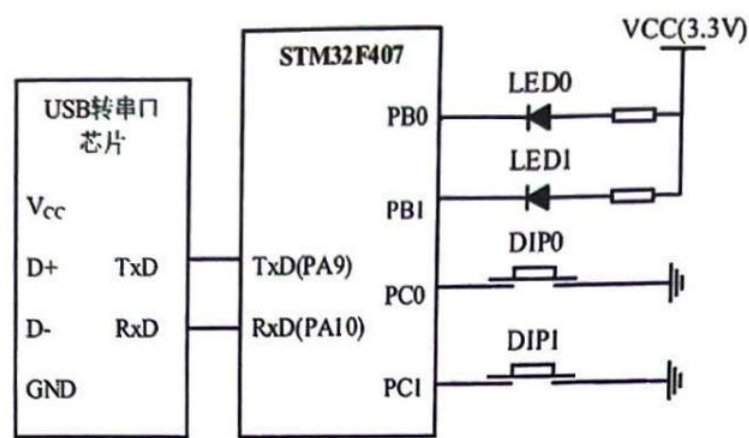


图 3 实验箱的 I/O 引脚连接示意图

附录 1：定时器控制 LED 亮灭相关代码

ST 官方库文件 stm32f10x.h 代码片段

```
1  #define PERIPH_BASE      ((uint32_t)0x40000000)
2  #define APB1PERIPH_BASE  PERIPH_BASE
3  #define APB2PERIPH_BASE  (PERIPH_BASE + 0x10000)
4  #define AHBPERIPH_BASE   (PERIPH_BASE + 0x20000)
5  #define GPIOA_BASE       (APB2PERIPH_BASE + 0x0800)
6  #define GPIOB_BASE       (AHBPERIPH_BASE + 0x0400)
7
8  typedef struct
9  {
10     __IO uint32_t CRL;
11     __IO uint32_t CRH;
12     __IO uint32_t IDR;
13     __IO uint32_t ODR;
14     __IO uint32_t BSRR;
15     __IO uint32_t BRR;
16     __IO uint32_t LCKR;
17 } GPIO_TypeDef;
```

ST 官方库文件 stm32f10x_gpio.c 代码片段

```
1  uint16_t GPIO_ReadInputData(GPIO_TypeDef* GPIOx)
2  {
3      return ((uint16_t)GPIOx->IDR);
4  }
5
6  void GPIO_WriteBit(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin,
7                      BitAction BitVal)
8  {
9      if (BitVal != Bit_RESET) {
10         GPIOx->BSRR = GPIO_Pin;
```

```
11     } else {  
12         GPIOx->BRR = GPIO_Pin;  
13     }  
14 }
```

主程序代码片段 (main.c)

```
1 // 此处略去 GPIO 初始化代码: 配置 PA2 为使能上拉电阻的输入、PA3 为推挽输出  
2 GPIO_WriteBit(GPIOA, GPIO_Pin_3, 0x00);  
3 while (1)  
4 {  
5     if (GPIO_ReadInputDataBit(GPIOA, GPIO_Pin_2) == 0) //①  
6     {  
7         GPIO_WriteBit(GPIOB, GPIO_Pin_3, 0x10);          //②  
8         TIM2_Delay();  
9     }  
10    GPIO_WriteBit(GPIOB, GPIO_Pin_3, 0x00);  
11 }  
12  
13 void TIM2_Delay()  
14 {  
15     TIM_TimeBaseInitTypeDef TIM_TimeBaseStructure;  
16     RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM2, ENABLE);  
17     TIM_TimeBaseStructure.TIM_Prescaler = 7999;  
18     TIM_TimeBaseStructure.TIM_Period = 999;  
19     TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up;  
20     TIM_TimeBaseInit(TIM2, &TIM_TimeBaseStructure);  
21     TIM_ClearFlag(TIM2, TIM_FLAG_Update);  
22     TIM_Cmd(TIM2, ENABLE);  
23     while (TIM_GetFlagStatus(TIM2, TIM_FLAG_Update) == RESET); //③  
24 }
```

附录 2: 中断控制 LED 的相关代码

ST 官方库文件 stm32f10x.h 代码片段

```
1 typedef enum IRQn  
2 {  
3     NonMaskableInt_IRQn    = -14, /*!< 2 Non Maskable Interrupt */  
4     MemoryManagement_IRQn = -12, /*!< 4 Cortex-M4 Memory Management Interrupt */  
5     EXTI0_IRQn             =  6,  /*!< EXTI Line0 Interrupt */  
6     EXTI1_IRQn             =  7,  /*!< EXTI Line1 Interrupt */  
7     EXTI2_IRQn             =  8,  /*!< EXTI Line2 Interrupt */  
8     EXTI3_IRQn             =  9,  /*!< EXTI Line3 Interrupt */  
9     /* 其他考题未涉及的中断号定义略去 */  
10 }
```

ST 官方库文件 stm32f10x_exti.h 代码片段

```
1  typedef struct
2  {
3      uint32_t EXTI_Line;          // Specifies the EXTI lines to be enabled or disabled.
4      EXTI_Mode_TypeDef EXTI_Mode; // Specifies the mode for the EXTI lines.
5      EXTI_Trigger_TypeDef EXTI_Trigger; // Specifies the trigger signal active edge.
6      FunctionalState EXTI_LineCmd;    // Specifies the new state of the selected lines.
7  } EXTI_InitTypeDef;
8
9  #define EXTI_Line0 ((uint32_t)0x000001) /*!< External interrupt line 0 */
10 #define EXTI_Line1 ((uint32_t)0x000002) /*!< External interrupt line 1 */
11 #define EXTI_Line2 ((uint32_t)0x000004) /*!< External interrupt line 2 */
12 #define EXTI_Line3 ((uint32_t)0x000008) /*!< External interrupt line 3 */
```

主程序代码片段 (main.c)

```
1  // 此处略去头文件信息
2  #define LED0(x) GPIO_WriteBit(GPIOB, GPIO_Pin_3, x) //①
3  #define DIPO GPIO_ReadInputDataBit(GPIOA, GPIO_Pin_2) //②
4
5  void LED0_Config(void);
6  void KEY0_Config(void);
7  void EXTI_Config(void);
8  void NVIC_Config(void);
9
10 int main(void)
11 {
12     // 略去 GPIO 相关配置
13     EXTI_Config();
14     NVIC_Config();
15     LED0(0);
16     while (1);
17 }
18
19 void EXTI_Config(void)
20 {
21     EXTI_InitTypeDef EXTI_InitStructure;
22     RCC_APB2PeriphClockCmd(RCC_APB2Periph_SYSCFG, ENABLE);
23     SYSCFG_EXTILineConfig(EXTI_PortSourceGPIOG, GPIO_PinSource4);
24     EXTI_InitStructure.EXTI_Line = ?; //③
25     EXTI_InitStructure.EXTI_Mode = EXTI_Mode_Interrupt;
26     EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger_Falling;
27     EXTI_InitStructure.EXTI_LineCmd = ENABLE;
28     EXTI_Init(&EXTI_InitStructure);
29 }
```

```
30 void NVIC_Config(void)
31 {
32     NVIC_InitTypeDef NVIC_InitStructure;
33     NVIC_PriorityGroupConfig(NVIC_PriorityGroup_1);
34     NVIC_InitStructure.NVIC_IRQChannel = ?;           //④
35     NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0;
36     NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
37     NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
38     NVIC_Init(&NVIC_InitStructure);
39 }
40
41 void EXTI2_IRQHandler(void)
42 {
43     if (EXTI_GetITStatus(EXTI_Line2) != RESET)
44     {
45         if (DIP0)
46             GPIOB->BSRR = 0x00000008;
47         else
48             GPIOB->BSRR = 0x00080000;
49         EXTI_ClearITPendingBit(EXTI_Line2);
50     }
51 }
```

附录 3：UART 通信相关代码

主程序代码片段（main.c）

```
1  #include "stm32f4xx.h"
2
3  void USART1_Config(unsigned int baud)
4  {
5      USART_InitTypeDef USART_InitStructure;
6      RCC_APB2PeriphClockCmd(RCC_APB2Periph_USART1, ENABLE);
7      USART_InitStructure.USART_BaudRate = baud;
8      USART_InitStructure.USART_WordLength = USART_WordLength_8b;
9      USART_InitStructure.USART_StopBits = USART_StopBits_2;
10     USART_InitStructure.USART_Parity = USART_Parity_No;
11     USART_InitStructure.USART_HardwareFlowControl =
12         USART_HardwareFlowControl_None;
13     USART_InitStructure.USART_Mode = USART_Mode_Rx | USART_Mode_Tx;
14     USART_Init(USART1, &USART_InitStructure);
15     USART_ClearFlag(USART1, USART_FLAG_TC);           //①
16     USART_Cmd(USART1, ENABLE);
17 }
18
19 int main(void)
20 {
21     unsigned char uc_send_ch = 0;
22     unsigned char uc_recv_ch = 0;
```

```
23 // GPIOA、GPIOB、GPIOC 初始化代码，此处略去
24 USART1_Config(115200);
25 while (1)
26 {
27     uc_send_ch = GPIOC->IDR & 0x0000 0003;           //②
28     ui_dip0_value; // unsigned int 类型转换为 unsigned char
29     if (USART_GetFlagStatus(USART1, USART_FLAG_TXE) == SET) //③
30         USART_SendData(USART2, uc_send_ch); // 向 USART2 发送数据
31     if (USART_GetFlagStatus(USART2, USART_FLAG_RXNE) != RESET) //④
32         uc_recv_ch = USART_ReceiveData(USART2); // 从 USART2 接收数据
33     //⑤此处添加代码
34 }
35 }
```

附录 4：STM32F103 系列 MCU 相关信息

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0

表 2 STM32F103 的 GPIOx_BSRR (x=A..G) 寄存器位定义

GPIO_BSRR[31:16] BRy: Port x reset bit y (y = 0…15)

BRy 为 1，表示 Port x 的输出数据寄存器 (ODRx) 的 y 位置 0，即 Port x 的 y 位输出 0。

GPIO_BSRR[15:0] BSy: Port x set bit y (y = 0…15)

BSy 为 1，表示 Port x 的输出数据寄存器 (ODRx) 的 y 位置 1，即 Port x 的 y 位输出 1。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0

表 3 STM32F103 的 GPIOx_BRR (x=A..G) 寄存器位定义

GPIO_BRR[15:0] BRy: Port x reset bit y (y = 0…15)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0

表 4 STM32F103 的 GPIO 输入端口 GPIOx_IDR (x=A..G) 寄存器位定义

IDR[15:0] IDRy: Port input data (y = 0…15)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved												MR19	MR18	MR17	MR16
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MR15	MR14	MR13	MR12	MR11	MR10	MR9	MR8	MR7	MR6	MR5	MR4	MR3	MR2	MR1	MR0

表 5 STM32F103 的中断屏蔽 EXTI_IMR 寄存器位定义

EXTI_IMR[19:0] MRx: Interrupt mask on line x

MRx 为 0，表示中断线 x 对应的中断被屏蔽，系统复位后默认各个位均为 0；MRx 为 1，表示中断线 x 对应的中断未被屏蔽。

Position	Priority	Type of priority	Acronym	Description	Address
-	-	-	-	Reserved	0x0000 0000
-	-3	fixed	Reset	Reset	0x0000 0004
-	6	settable	SysTick	System tick timer	0x0000 003C
0	7	settable	WWDG	Window Watchdog interrupt	0x0000 0040
10	17	settable	EXTI4	EXTI Line0 interrupt	0x0000 0068
23	30	settable	EXTI9_5	EXTI Line[9:5] interrupts	0x0000 009C
28	35	settable	TIM2	TIM2 global interrupt	0x0000 00B0
37	44	settable	USART1	USART1 global interrupt	0x0000 00D4
38	45	settable	USART2	USART2 global interrupt	0x0000 00D8
39	46	settable	USART3	USART3 global interrupt	0x0000 00DC

表 6 STM32F103 系列 MCU 的中断向量表（部分）

附录 5：ARM 汇编指令相关信息

ADD 指令

加法指令，典型语法是“ADD Rd, Rn, Rm”，该指令将 Rn + Rm 的结果存入 Rd。

示例 1: ADD R1, R2, R3 示例 2: ADD R1, R1, R2

功能解析: R1 = R2 + R3 功能解析: R1 = R1 + R2

LDR 指令

从存储器读取字数据至寄存器。典型语法为“LDR Rd, [Rn, #offset]”，该指令从存储器位置 Rn + offset 读取一个字存入寄存器 Rd。若指定操作数选址方式为前变址寻址（从位置 Rn + offset 读取一个字，更新 Rn 为 Rn + offset），其典型语法为“LDR Rd, [Rn, #offset]!”。若指定操作数选址方式为后变址寻址（从位置 Rn 读取一个字，更新 Rn 为 Rn + offset），其典型语法为“LDR Rd, [Rn], #offset”。

示例 1: LDR R3, [R4]

功能解析: R4 中保存的是一个字数据的地址，读取该地址起始的 4 个字节至 R3 寄存器。

示例 2: LDR R3, [R4, #0x8]

功能解析: 操作数（字数据）的地址是 R4 中数值加 0x8，读取该操作数（字数据）至 R3 寄存器。

示例 3: LDR R3, [R4, #0x8]!

功能解析: 先将 R4 中数值加 0x8，再把 R4 中数值作为操作数（字数据）的地址，读取该操作数（字数据）至 R3 寄存器。

2025-2026 学年第 2 学期期末试卷

中国科学技术大学信息科学技术学院
2025-2026 学年第 2 学期考试试卷

计 算 机 原 理 与 嵌 入 式 系 统

课程编号：EE2003 考试形式：半开卷

姓名_____ 学号_____ 班级_____

☒ A 卷 ☐ B 卷

题号	一	二	三	四 ()	四 ()	四 ()	四 ()	总分
得分								

* 第四题中选做的题号请填入上表。

第二题（选择题）答题卡

11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30

第三题（判断题）答题卡

31	32	33	34	35	36	37	38	39	40
41	42	43	44	45					

一、填空题（15 分，每小题 1 分，直接在试卷上答题）

（一）数据的表示和存取

本课程中实验箱系统采用 STM32F407 芯片，考虑该系统上运行的 C 程序。程序中某变量保存在寄存器 R1，R1 内容为 0xFFFF FFF8。综合运用课程知识，回答如下问题。

1. 若该变量是 32 位无符号整数，变量的真值为_____，若该变量经饱和（Saturation）操作得到 16 位无符号整数，该 16 位无符号数的十进制真值为_____。
注：饱和运算，即运算结果大于（数表示法所能表示的）上限或小于下限时，结果就等于上限或是下限。
2. 若该变量是 32 位有符号整数，变量的十进制真值为_____。
3. 若 R1 中数值是一个 16 位有符号整数经符号扩展（Sign Extension）转换至 32 位得到，则该 16 位有符号整数的 16 进制机器数为_____，该 16 位有符号整数的十进制真值为_____。
注：有符号整数进行符号扩展时，扩展的位（即新增的位）将复制原始数值的符号位。

(二) Cortex-M4 内核与存储器之间的数据传输

已知某 STM32F407 系统片内仅实现了 48kB 的 SRAM（起始地址 0x2000 0000），以及 64kB 的 Flash（起始地址 0x0800 0000），且无外接存储器。系统支持位段操作，即地址 0x2000 0000 起始的 1MB 会被映射至地址 0x2200 0000 起始的别名区域。图 1 为该系统总线上传输的某个片段；表 1 为某程序执行过程中图 1 操作开始前存储器内存储的部分信息。综合运用课程知识，并参考“附录 5：ARM 汇编指令相关信息”，回答下列问题（提示：数值用 16 进制表示）。

4. 该系统片上 SRAM 可有效寻址的地址范围是_____。
5. 该系统片上 Flash 可有效寻址的地址范围是_____。
6. 上电复位后执行的第一条指令的地址是_____。
7. 若启动代码中设置主栈大小为 256B。
该栈在 4GB 存储空间所占用的地址范围是_____；
第一次压栈的数据值是_____。
8. 参考附录 5 中 LDR 指令格式，若数据 B 相关的总线操作由指令“LDR R1, [R0]”引起，则该指令执行前 R0 保存的数值是_____；该指令执行后 R1 保存的数值是_____。
9. 若数据 D 的传输由指令“LDR R1, [R0]”触发：
该指令执行前 R0 中保存的数值是_____，
该指令执行后 R1 中保存的数值是_____。
10. 若将地址 0x2000 0008 起始的字数据保存到 R4 中，再执行指令“LDR R3, [R4]”，会触发_____。

表 1 AHB 总线传输发生前 STM32F407 片内存储器信息

地址	字数据
0x0000 0000	0x2000 1000
0x0000 0004	0x0800 0100
0x0000 0008	0x0800 2000
...	
0x2000 0000	0xFF00 2025
0x2000 0004	0xFFFF 2026
0x2000 0008	0x00FF 2027
...	
0x2000 1000	0x1234 2023
0x2000 1004	0x2025 2026
0x2000 1008	0x2026 2027

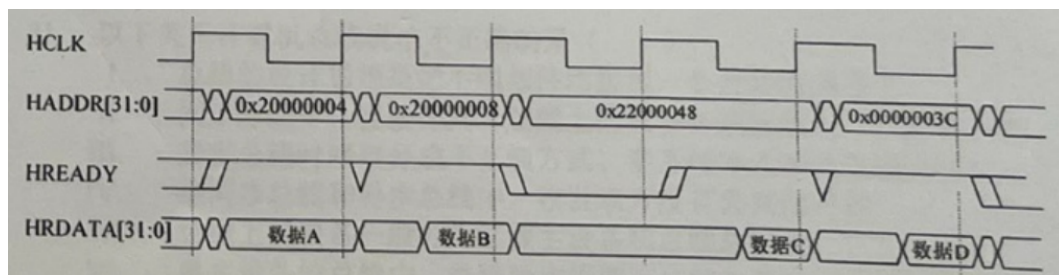


图 1 某 Cortex-M 系统 AHB 总线上的数据传输片段

二、单项选择题（30 分，每小题 1.5 分，在试卷首页答题卡答题）

11. 依据 IEEE-754 标准，十进制数 2026D 表示为单精度浮点数是（ ）
 - A. 0x44FD4000
 - B. 0x44F4000
 - C. 0xE4FD4000
 - D. 0xF4FD4000
12. 以下关于 2026H、0010 0000 0010 0100B、2026D 三个数描述正确的有（ ）
 - I. 若三个数均为无符号数，则最大的数是 2026H
 - II. 若三个数均为无符号数，则最大的数是 0010 0000 0010 0100B
 - III. 若三个数均为无符号数，则最大的数是 2026D
 - IV. 若三个数均为无符号数，则最小的数是 2026H
 - V. 若三个数均为无符号数，则最小的数是 0010 0000 0010 0100B
 - VI. 若三个数均为无符号数，则最小的数是 2026D
 - A. I、V
 - B. II、VI
 - C. I、VI
 - D. II、V
 - E. III、VI
 - F. A-E 均不合题意
13. 某 Cortex-M 系统中，带符号整数用补码表示。变量 a、b、c、d 的机器数分别为 0x7FFFFFFF、0x0000000F、0xFFFFFFFF0、0x80000001，下列哪些运算会导致程序状态寄存器 PSR 的溢出位 $V = 1$ （ ）
 - A. a-b 运算
 - B. a+c 运算
 - C. b+c 运算
 - D. b+d 运算
 - E. c+d 运算
 - F. A-E 均不合题意
14. 在多级存储体系中，缓存——主存结构的作用是解决（ ）问题。
 - A. 主存与 CPU 速度不匹配
 - B. 辅存与 CPU 速度不匹配
 - C. 主存和辅存速度不匹配
 - D. 主存容量不足
 - E. 主存和辅存容量不匹配
 - F. A-E 均不合题意

15. 以下关于流水线相关冲突说法不正确的有 ()

- I. 数据相关指一条指令需要使用前一条（尚未完成计算）指令的计算结果
 - II. 多个指令同时需要使用同一个寄存器，这属于数据相关冲突
 - III. 程序中的转移指令有可能引起控制相关
 - IV. 控制相关冲突可以通过插入“气泡”的方式消除
 - V. 数据相关冲突可以通过插入“气泡”的方式消除
 - VI. 资源相关冲突可以通过插入“气泡”的方式消除
 - VII. 动态转移预测算法常用来解决控制相关冲突
- A. I、IV B. II、IV C. II、VI D. III、IV
- E. IV、VI F. A-E 均不合题意

16. 以下关于 Cortex-M3/M4 流水线说法正确的有 ()

- I. 流水线机制能够有效提高 IPC (Instructions Per Cycle)
 - II. 流水线机制能够有效提高 CPI (Cycles Per Instruction)
 - III. 流水线分成取指令、指令解码、指令执行三个阶段
 - IV. 流水线分成取指令、指令解码、取操作数、指令执行、回写结果五个阶段
 - V. 流水线中每个阶段的执行时间应取各个子阶段执行时间的最大值
 - VI. 流水线中每个阶段的执行时间应取各个子阶段执行时间的最小值
 - VII. 流水线中每个阶段的执行时间应取各个子阶段执行时间的平均值
- A. I、III、V B. I、III、VI C. II、III、VI D. II、III、V
- E. II、IV、V F. A-E 均不合题意

17. 以下哪些常用于 MCU 系统的辅助存储接口 ()

- I. SCSI
 - II. IDE
 - III. SATA
 - IV. I²C
 - V. SPI
 - VI. UART
 - VII. USART
 - VIII. PCIE
- A. I、II B. II、III C. III、V D. IV、VI
- E. V、VI F. A-E 均不合题意

18. 以下关于缓存（Cache）说法正确的是（ ）

- I. 全相联映射方式的缓存命中率最低
- II. 全相联映射方式的硬件电路最为复杂
- III. 直接相联映射方式的缓存命中率一般会低于组相联映射方式
- IV. 组相联映射方式中，Cache 组内的块数一般取值较小（如 16）
- V. 组相联映射方式中，Cache 的组数目一般取值较小（如 16）
- VI. STM32 单片机采用了直接相联映射方式
- VII. PC 上的通用 CPU 一般会采用直接相联映射方式

- A. I、II、III B. I、II、IV C. II、III、IV D. II、III、V
E. II、III、VII F. A-E 均不合题意

19. 以下关于计算机存储器描述正确的是（ ）

- I. 内存即主存储器
- II. 内存和外存使用相同的地址空间
- III. 缓存地址空间和主存地址空间是相同的地址空间
- IV. TCM（Tightly Coupled Memory，紧耦合内存）和主存使用相同的地址空间
- V. 虚存空间的大小通常就是 CPU 可寻址的最大范围
- VI. 存储器芯片的地址线 and 数据线是不可以复用的

- A. I、III、IV B. I、II、V C. I、IV、V D. III、IV、V
E. III、IV、VI F. A-E 均不合题意

20. 以下关于“地址”描述不正确的是（ ）

- I. 地址总线中信号线数目影响了可寻址空间的大小
- II. 存储器地址可以用来标识存储器内的存储单元
- III. I/O 端口地址和存储器地址必须使用相同的地址线
- IV. I/O 端口地址和存储器地址必须使用独立的地址线
- V. 缓存（Cache）地址空间和主存地址空间是相同的地址空间
- VI. 紧耦合内存（TCM）地址空间和主存地址空间是相同的地址空间
- VII. 虚存空间中的虚拟地址可以转换为物理地址

- A. I、III、IV B. II、III、V C. III、IV、VI D. III、IV、V
E. III、V、VI F. A-E 均不合题意

21. 以下关于计算机总线说法不正确的是 ()

- I. 总线的设计思想是把不同部件均接到一个公共的通道上
- II. 同步总线中, 挂接在同一总线上的设备必须按照统一的时钟工作
- III. 异步总线时序可分成不互锁方式、半互锁方式和全互锁方式
- IV. 半同步总线和异步总线中, 收发双方没有公共的时钟
- V. 总线上的设备一般分为总线主设备和总线从设备
- VI. 单主设备的总线中, 总线操作周期可分解为寻址子周期和数据传送子周期
- VII. 无论是单主设备环境还是多主设备环境均需要总线仲裁

- A. II、IV B. II、VII C. IV、VII D. V、VII
E. VI、VII F. A-E 均不合题意

22. 以下关于 AHB 总线描述不正确的有 ()

- I. AHB 总线支持多主设备
- II. AHB 的读数据和写数据复用同一组信号线
- III. AHB 通过分离读地址通道和写地址通道来提升性能
- IV. AHB 总线采用了同步总线时序
- V. AHB 总线中插入等待周期的数据传输, 本质上是一种半同步时序
- VI. AHB 中, “驱动地址” 和 “驱动数据” 可构成两级 “流水线” 操作
- VII. AHB 不提供任何与电气特征有关的信息

- A. II、III B. III、IV C. III、V D. III、VI
E. V、VII F. A-E 均不合题意

23. 以下关于 AHB 突发传输描述不正确的有 ()

- I. 突发传输可以通过 HSIZE[2:0] 指定传输数据块的长度
- II. 突发传输通过 HTRANS[1:0] 指示传输状态, 如 “SEQ”、“IDLE” 等
- III. 突发传输在传输开始前不用指定传输数据的长度
- IV. WRAP4 类型的突发传输地址采用循环递增方式
- V. INCR8 类型的突发传输支持的传输数据块长度必须是 8 个字节
- VI. WRAP4 类型的突发传输支持的传输数据块长度必须是 4 个字

- A. I、II、III B. I、II、IV C. II、III、IV D. II、IV、V
E. I、V、VI F. A-E 均不合题意

24. 某使能了位段操作的 Cortex-M4 系统, 存储空间按照字节编址, 那么一个对准存放的 float 类型临时变量(字数据)的起始地址不会是()

- I. 0x0000 0000
- II. 0x2026 0000
- III. 0xFF00 2026
- IV. 0x2200 2026
- V. 0x2026 0004
- VI. 0x2026 0008

- A. I、II、III B. I、III、IV C. II、III、IV D. II、V、VI
E. III、IV、V F. A-E 均不合题意

25. 以下关于实验所用 STM32F407 系列处理器描述不正确的是()

- I. 复位后默认是非特权访问等级
- II. 非特权访问等级可以访问 4GB 空间的所有区域
- III. 复位后使用主栈指针 MSP
- IV. 具有处理模式和线程模式两种操作模式
- V. 处理模式一定具有特权访问等级
- VI. 线程模式一定具有特权访问等级

- A. I、II、III B. I、III、IV C. I、II、VI D. II、IV、V
E. IV、V、VI F. A-E 均不合题意

26. 以下关于 Cortex-M3/M4 存储器子系统说法不正确的是()

- I. 既可以配置为大端模式, 也可以配置为小端模式
- II. 地址段 0x2000 0000--0x0001 FFFF 可以存放代码
- III. 地址段 0x2000 0000--0x0001 FFFF 不能存放代码
- IV. SRAM 区用于存放数据, 不能存放代码
- V. 外部 RAM 区用于存放数据, 也可以存放代码
- VI. 处理器读存储器时, 存储器系统会检查所访问区域的存储器访问属性
- VII. SRAM 区有 1MB 位段区被映射为 16MB 位带别名区域

- A. I、IV、V B. II、III、IV C. III、IV、V D. III、IV、VI
E. III、IV、VII F. A-E 均不合题意

27. 以下关于 Cortex-M3/M4 栈说法正确的是 ()

- I. 具有双栈的结构, 有 MSP 和 PSP 两个堆栈指针
- II. 栈的访问一定是字对齐的
- III. 栈的访问需要特权访问等级
- IV. 非特权线程模式只能使用 MSP 而不能使用 PSP
- V. 往栈内放置一个新的数据后, 栈指针数值会增大
- VI. 往栈内放置一个新的数据后, 栈指针数值会减小
- VII. 栈空间只能配置在 4GB 空间的 SRAM 区

- A. I、II、III B. I、II、IV C. I、II、V D. I、II、VI
- E. II、VI、VII F. A-E 均不合题意

28. 以下关于 STM32 处理器中 “I/O 端口地址” 描述正确的有 ()

- I. “I/O 端口地址” 和 “存储器地址” 是独立编址的, 对应不同的访问指令
- II. GPIOA 模块中输入寄存器 IDR 的地址属于 “I/O 端口地址”
- III. GPIOA 模块中输出寄存器 ODR 的地址属于 “I/O 端口地址”
- IV. 定时器模块中的控制寄存器 CR1、CR2 的地址不属于 “I/O 端口地址”
- V. USART1 模块中的状态寄存器 SR 的地址不属于 “I/O 端口地址”
- VI. SPI 模块 CR1、CR2 寄存器的地址不属于 “I/O 端口地址”

- A. I、II B. I、IV C. II、III D. III、V
- E. V、VII F. A-E 均不合题意

29. 以下关于 Cortex-M3/M4 中断 (异常) 子系统说法正确的有 ()

- I. 中断向量表必须存放在 4GB 存储空间的代码区
- II. 一般情况下, 中断向量表设计为保存中断服务子程序的入口地址
- III. 中断向量表中不同的行存放的字数据不可能是相同的
- IV. 子优先级高的异常可以打断子优先级低的异常
- V. 组优先级高的异常可以打断组优先级低的异常
- VI. 上电复位后, 取出地址为 0x00000000 的第一个字, 作为复位中断向量

- A. I、II B. II、III C. II、IV D. II、V
- E. IV、VI F. A-E 均不合题意

30. 以下关于 STM32 中 USART 模块说法正确的是 ()

- I. 波特率可以在通信过程中协商
- II. 可以挂接多个接收节点, 不同接收节点通过器件地址予以区分
- III. RXNE 标志位为 1 表示数据接收寄存器空
- IV. TXE 标志位为 1 表示数据移位寄存器空
- V. 可以在接收到数据时产生中断
- VI. 可以在发送数据寄存器空时产生中断

- A. I、II B. II、III C. III、IV D. IV、V
E. V、VI F. A-E 均不合题意

三、判断题 (15 分, 每小题 1 分, 打“√”或“×”, 在试卷首页答题卡答题)

31. STM32F103 系列处理器的 SYSCLK 可以来源于 HSE、LSE、HSI 或 LSI。()
32. 独立看门狗模块的时钟由 HSI 提供时钟。()
33. DDR4 内存采用了 8bit 的数据预取 (8n-Prefetch)。()
34. Cortex-M3 系统中, MSP 的值不能设置为 0x2026 2026。()
35. Cortex-M3 系统中, 非特权等级代码不能访问异常的优先级寄存器。()
36. 每一条机器指令可以由一段用微指令编成的微程序来解释执行。()
37. AHB 的流水线 Split 机制允许总线操作的地址阶段和数据阶段分裂开来。()
38. Cortex-M 处理器中, 先执行的指令不一定先完成。()
39. 若 GPIOA 的 ODR 寄存器的地址是 0x4001080C, 那么代码 “GPIOA->ODR=0xFFFF” 等价于 C 代码 “*(volatile unsigned long*)(0x4001080C)=0xFFFF”。()
40. 一个指令周期通常包括多个总线周期。()
41. STM32F407 处理器的每一根外部中断线都各自对应一个不同的中断向量。()
42. 计算机中的 “ μ arch” 通常对应 ISA 的硬件实现方式。()
43. 在多主控设备的情形下总线周期必须包括仲裁阶段。()
44. Cortex-M 的位段操作可简化操作但仍无法解决 “读 → 修改 → 写” 问题。()
45. Cortex-M3/M4 存储器子系统只能配置为小端模式。()

四、设计及综合题（40 分，每题 10 分，在稿纸上答题）

（一）CPU 连接存储器

46.（10 分）

某系统采用 STM32F103ZE 处理器（Cortex-M3，32 根地址线，32 根数据线），存储器按字节编址。现有 2 片 SRAM 芯片（记为芯片 x 和芯片 y），规格为 256K × 16-bit。试为该系统设计片外扩展 SRAM 存储器子系统，要求 2 片 SRAM 芯片处于连续地址范围，且起始地址为外部 RAM 区的起始地址 0x6000 0000。

- （1）（1 分）写出片外扩展 SRAM 存储子系统的容量。
- （2）（2 分）写出片外扩展 SRAM 存储子系统所占用的地址范围。
- （3）（3 分）画出地址线、数据线及片选信号的连接示意图。
- （4）（2 分）假设该片外扩展 SRAM 存储子系统的起始地址为“A”，处理器读取地址“A”的字数据得到“0x12345678”，该字数据的 4 个字节分别来自哪块芯片？
- （5）（2 分）能否以位段操作的方式访问该片外扩展 SRAM 存储子系统的某个比特，并解释原因。

（二）Cache 及其应用

47.（10 分）

某计算机系统主存地址空间大小为 1MB，按字节编址。指令 Cache 和数据 Cache 分离，容量均为 16kB，均采用 16 路组相联映射方式，每个 Cache 行（cache line）大小为 256B。现有两个功能相同的程序 1 和程序 2，其伪代码如下所示。

程序 1

```
int a[64][64];
int sum_array1() {
    int i, j, sum=0;
    for(i=0; i<64; i++)
        for(j=0; j<64; j++)
            sum += a[i][j];
    return sum;
}
```

程序 2

```
int a[64][64];
int sum_array1() {
    int i, j, sum=0;
    for(j=0; j<64; j++)
        for(i=0; i<64; i++)
            sum += a[i][j];
}
```

假定 `int` 类型数据用 32 位补码表示，程序编译时 `i`、`j`、`sum` 均分配在寄存器中，数组 `a` 按行优先方式存放，其首地址为 `0x0`，代码运行前 Cache 为空。请回答下列问题，要求说明理由或给出计算过程。

- (1) (2 分) 数据 Cache 被分为多少组？每组内多少个 Cache 行？
- (2) (2 分) 主存被分为多少页？每页内多少个 Cache 行？
- (3) (2 分) 数组 `a` 能否全部载入 Cache？
- (4) (2 分) 数组元素 `a[1][31]` 载入 Cache 时，写出其对应的 Cache 地址范围。
- (5) (2 分) 计算程序 1 和程序 2 的数据访问命中率（写出表达式即可）。

(三) GPIO 及定时器应用

48. (10 分)

在实验箱上用定时器模块结合输出比较模块实现 LED0 亮灭控制，硬件连接如图 2 所示。拟设计的程序功能为：使 LED0 呈现亮 1.5 秒、灭 0.5 秒的呼吸灯效果。部分代码如附录 1 所示。已知该系统配置为使用外部输入的 8MHz 时钟源且未使能 PLL，AHB 预分频系数、APB1 预分频系数、TIM2 倍频系数均设置为 1。请结合课程实验回答以下问题。

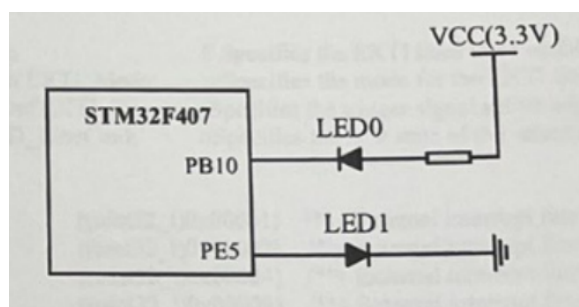


图 2 实验箱的 I/O 引脚连接示意图

- (1) (4 分) 解释 `main.c` 中代码行①、②、③、⑥的含义。
- (2) (4 分) 补充完整定时函数的空余部分④和⑤。
- (3) (2 分) 代码行⑦中 “`my_ccr3_value`” 的值应配置为什么？

(四) GPIO 及中断

49. (10 分)

考虑一个 STM32F103 系统，在该系统上执行“附录 2：中断控制 LED 的相关代码”所示代码，能够实现拨动开关 DIP0 由不接地（引脚对应位 = 1）拨至接地（引脚对应位 = 0）时，触发外部中断，在其中断服务子程序中熄灭 LED 灯；且拨动开关 DIP1 由不接地（引脚对应位 = 1）拨至接地（引脚对应位 = 0）时，触发外部中断，在其中断服务子程序中点亮 LED 灯。回答下列问题。

- (1) (4 分) 根据代码，画出最小系统示意图，并要求画出时钟、复位以及 LED、DIP0 和 DIP1 相关引脚的连接方式。
- (2) (1 分) 补全函数 `EXTI_Config()` 中①行的代码。

- (3) (1 分) 补全函数 `NVIC_Config()` 中②行的代码。
- (4) (1 分) 补全解释③行代码的功能。
- (5) (2 分) 补全函数 `EXTI3_IRQHandler()` 中④行的代码。
- (6) (1 分) 该系统是否会形成中断嵌套？并解释原因。

附录 1：定时器结合输出比较控制 LED 亮灭相关代码

```

1 void RCC_Configuration() {
2     RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOE, ENABLE); // ①
3     RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOB, ENABLE);
4     RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM2, ENABLE); // ②
5 }
6
7 void GPIO_Configuration() {
8     GPIO_PinAFConfig(GPIOB, GPIO_PinSource10, GPIO_AF_TIM2); // ③
9     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10;
10    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz;
11    GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
12    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;
13    GPIO_Init(GPIOB, &GPIO_InitStructure);
14
15    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_5;
16    GPIO_InitStructure.GPIO_Mode = GPIO_OType_PP;
17    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz;
18    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;
19    GPIO_Init(GPIOE, &GPIO_InitStructure);
20 }
21
22 void TIM2_Configuration() {
23     TIM_TimeBaseInitTypeDef TIM_TimeBaseStructure;
24     TIM_TimeBaseStructure.TIM_Prescaler = ④;
25     TIM_TimeBaseStructure.TIM_Period = ⑤;
26     TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up;
27     TIM_TimeBaseStructure.TIM_ClockDivision = TIM_CKD_DIV1; // ⑥
28     TIM_TimeBaseInit(TIM2, &TIM_TimeBaseStructure);
29 }
30
31 void PWM3_Configuration() {
32     TIM_OCInitTypeDef TIM_OCInitStructure;
33     // TIM2_CNT<TIM2_CCR1时输出高电平,
34     // TIM2_CCR1<TIM2_CNT<TIM2_ARR时输出低电平
35     TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM1;
36     TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable;
37     TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_High;
38     TIM_OC3Init(TIM2, &TIM_OCInitStructure);
39     TIM_OC3PreloadConfig(TIM2, TIM_OCPreload_Enable);
40     TIM_ARRPreloadConfig(TIM2, ENABLE);
41     TIM_SetCompare3(TIM2, my_ccr3_value); // ⑦ 设置捕获/比较寄存器CCR的值

```

```
42     TIM_Cmd(TIM2, ENABLE);
43 }
44
45 int main() {
46     RCC_Configuration();
47     GPIO_Configuration();
48     TIM2_Configuration();
49     PWM3_Configuration();
50     while (1) {}
51 }
```

附录 2：中断控制 LED 的相关代码

ST 官方库文件 stm32f10x.h 代码片段

```
1  typedef enum IRQn
2  {
3      NonMaskableInt_IRQn    = -14, /*!< 2 Non Maskable Interrupt */
4      MemoryManagement_IRQn = -12, /*!< 4 Cortex-M4 Memory Management Interrupt */
5      EXTI0_IRQn = 6,             /*!< EXTI Line0 Interrupt */
6      EXTI1_IRQn = 7,             /*!< EXTI Line1 Interrupt */
7      EXTI2_IRQn = 8,             /*!< EXTI Line2 Interrupt */
8      EXTI3_IRQn = 9,             /*!< EXTI Line3 Interrupt */
9      /* 其他考题未涉及的中断号定义略去 */
10 }
```

ST 官方库文件 stm32f10x_exti.h 代码片段

```
1  typedef struct
2  {
3      uint32_t EXTI_Line;          /* Specifies the EXTI lines to be enabled or disabled. */
4      EXTI_Mode_TypeDef EXTI_Mode; /* Specifies the mode for the EXTI lines. */
5      EXTI_Trigger_TypeDef EXTI_Trigger; /* Specifies the trigger signal active edge for the EXTI lines. */
6      FunctionalState EXTI_LineCmd; /* Specifies the new state of the selected EXTI lines. */
7  } EXTI_InitTypeDef;
8
9  #define EXTI_Line0 ((uint32_t)0x000001) /*!< External interrupt line 0 */
10 #define EXTI_Line1 ((uint32_t)0x000002) /*!< External interrupt line 1 */
11 #define EXTI_Line2 ((uint32_t)0x000004) /*!< External interrupt line 2 */
12 #define EXTI_Line3 ((uint32_t)0x000008) /*!< External interrupt line 3 */
```

主程序代码片段 (main.c)

```
1 // 此处略去头文件信息
2 void LED0_Config(void);
3 void KEY0_Config(void);
4 void KEY1_Config(void);
5 void EXTI_Config(void);
6 void NVIC_Config(void);
7
8 int main(void)
9 {
10     // 略去 GPIO 相关配置
11     EXTI_Config();
12     NVIC_Config();
13     LED0(0);
14     while (1);
15 }
16
17 void EXTI_Config(void)
18 {
19     EXTI_InitTypeDef EXTI_InitStructure;
20     RCC_APB2PeriphClockCmd(RCC_APB2Periph_SYSCFG, ENABLE);
21     SYSCFG_EXTILineConfig(EXTI_PortSourceGPIOA, GPIO_PinSource2);
22     EXTI_InitStructure.EXTI_Line = ?; // ①
23     EXTI_InitStructure.EXTI_Mode = EXTI_Mode_Interrupt;
24     EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger_Falling;
25     EXTI_InitStructure.EXTI_LineCmd = ENABLE;
26     EXTI_Init(&EXTI_InitStructure);
27
28     SYSCFG_EXTILineConfig(EXTI_PortSourceGPIOA, GPIO_PinSource3);
29     EXTI_InitStructure.EXTI_Line = ?;
30     EXTI_InitStructure.EXTI_Mode = EXTI_Mode_Interrupt;
31     EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger_Falling;
32     EXTI_InitStructure.EXTI_LineCmd = ENABLE;
33     EXTI_Init(&EXTI_InitStructure);
34 }
35
36 void NVIC_Config(void)
37 {
38     NVIC_InitTypeDef NVIC_InitStructure;
39     NVIC_PriorityGroupConfig(NVIC_PriorityGroup_1);
40     NVIC_InitStructure.NVIC_IRQChannel = ?; // ②
41     NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0;
42     NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
43     NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
44     NVIC_Init(&NVIC_InitStructure);
45
46     NVIC_InitStructure.NVIC_IRQChannel = ?;
47     NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 1;
```

```
48     NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
49     NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
50     NVIC_Init(&NVIC_InitStructure);
51 }
52
53 void EXTI2_IRQHandler(void)
54 {
55     if (EXTI_GetITStatus(EXTI_Line2) != RESET)
56     {
57         GPIOB->BSRR = 0x04000000; // ③
58         EXTI_ClearITPendingBit(EXTI_Line2);
59     }
60 }
61
62 void EXTI3_IRQHandler(void)
63 {
64     if (EXTI_GetITStatus(EXTI_Line3) != RESET)
65     {
66         // ④ 补充此处代码
67         EXTI_ClearITPendingBit(EXTI_Line3);
68     }
69 }
```

附录 4：STM32F103 系列 MCU 相关信息

表 2 STM32F103 的 GPIOx_BSRR (x = A..G) 寄存器位定义

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0

GPIO_BSRR[31:16] BRy: Port x reset bit y (y=0..15)
BRy 为 1，表示 Port x 的输出数据寄存器 (ODRx) 的 y 位置 0，即 Port x 的 y 位输出 0。
GPIO_BSRR[15:0] BSy: Port x set bit y (y=0..15)
BSy 为 1，表示 Port x 的输出数据寄存器 (ODRx) 的 y 位置 1，即 Port x 的 y 位输出 1。

表 3 STM32F103 的 GPIOx_BRR (x = A..G) 寄存器位定义

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0

GPIO_BRR[15:0] BRy: Port x reset bit y (y=0..15)

表 4 STM32F103 的 GPIO 输入端口 GPIOx_IDR (x = A..G) 寄存器位定义

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0

IDR[15:0] IDRy: Port input data (y = 0..15)

表 5 STM32F103 的中断屏蔽 EXTI_IMR 寄存器位定义

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved												MR19	MR18	MR17	MR16
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MR15	MR14	MR13	MR12	MR11	MR10	MR9	MR8	MR7	MR6	MR5	MR4	MR3	MR2	MR1	MR0

EXTI_IMR[19:0] MRx: Interrupt mask on line x

MRx 为 0，表示中断线 x 对应的中断被屏蔽，系统复位后默认各个位均为 0；MRx 为 1，表示中断线 x 对应的中断未被屏蔽。

表 6 STM32F103 系列 MCU 的中断向量表（部分）

Position	Priority	Type of priority	Acronym	Description	Address
—	—	—	—	Reserved	0x0000_0000
—	—3	fixed	Reset	Reset	0x0000_0004
—	6	settable	SysTick	System tick timer	0x0000_003C
0	7	settable	WWDG	Window Watchdog interrupt	0x0000_0040
8	15	settable	EXTI2	EXTI Line2 interrupt	0x0000_0060
9	16	settable	EXTI3	EXTI Line3 interrupt	0x0000_0064
10	17	settable	EXTI4	EXTI Line4 interrupt	0x0000_0068
23	30	settable	EXTI9_5	EXTI Line[9:5] interrupts	0x0000_009C
28	35	settable	TIM2	TIM2 global interrupt	0x0000_00B0
37	44	settable	USART1	USART1 global interrupt	0x0000_00D4
38	45	settable	USART2	USART2 global interrupt	0x0000_00D8

附录 5：ARM 汇编指令相关信息

ADD 指令

加法指令，典型语法是“ADD Rd, Rn, Rm”，该指令将 $Rn + Rm$ 的结果存入 Rd。

示例 1: ADD R1, R2, R3

功能解析: $R1 = R2 + R3$ 。

示例 2: ADD R1, R1, R2

功能解析: $R1 = R1 + R2$ 。

LDR 指令

从存储器读取字数据至寄存器。典型语法为“LDR Rd, [Rn, #offset]”，该指令从存储器位置 $Rn + offset$ 读取一个字存入寄存器 Rd。若指定操作数选址方式为前变址寻址（从位置 $Rn + offset$ 读取一个字，更新 Rn 为 $Rn + offset$ ），其典型语法为“LDR Rd, [Rn, #offset]！”。若指定操作数选址方式为后变址寻址（从位置 Rn 读取一个字，更新 Rn 为 $Rn + offset$ ），其典型语法为“LDR Rd, [Rn], #offset”。

示例 1: LDR R3, [R4]

功能解析: R4 中保存的是一个字数据的地址，读取该地址起始的 4 个字节至 R3 寄存器。

示例 2: LDR R3, [R4, #0x8]

功能解析: 操作数（字数据）的地址是 R4 中数值加 0x8，读取该操作数（字数据）至 R3 寄存器。

示例 3: LDR R3, [R4, #0x8]！

功能解析: 先将 R4 中数值加 0x8，再把 R4 中数值作为操作数（字数据）的地址，读取该操作数（字数据）至 R3 寄存器。

校核说明

本册依据所提供的三份原卷逐页核对后重排。2023、2025 年原卷分别有 13、14 页；2026 年原 PDF 的 7 张照片覆盖印刷第 1–13 页。题目、选项、代码、图表及附录均以原卷可辨内容为依据。

试卷年份	题号范围	编号题图	附录编号
2023	1–49	图 1–3	1–5
2025	1–50	图 1–3	1–5
2026	1–49	图 1–2	1、2、4、5

本次校订

恢复了旧整理稿中与原卷不符的否定词、选项组合、地址和代码行；补齐寄存器位定义中的遗漏。部分电路图、时序图及浮点位域图从原 PDF 重新完整裁取，保留图内信号名、引脚、边框和位号。图表使用原卷编号，题干和选项保持在同一页。

原卷保留事项

- 以下内容可在原卷中直接确认，因此照录；使用时应结合授课教师的正式勘误。
- (1) 2023 年首页上部标题被裁切或涂黑，本册以可核对的学年、课程及考试形式整理卷首，未补写不可辨的院校抬头。附录代码中的缺分号、头文件及 USART 编号混用等，均保留原文。
 - (2) 2025、2026 年第 26 题的 II、III 均出现结束地址小于起始地址的写法：0x2000 0000 至 0x0001 FFFF。旧稿曾改动结束地址，本次恢复扫描原文。
 - (3) 2025 年第 48 题文字给定 8MHz 时钟，图 2 标为 16MHz；附录 1 的 PA3、GPIOA 与图中 PB3 及循环内 GPIOB 不一致，附录 2 的引脚映射与题目描述也有差异，均照录。
 - (4) 2026 年原卷附录编号由 2 跳至 4，并未提供附录 3；第 28 题 E 选项引用了该题未列出的 VII。均保留原编号和原选项。

校核日期：2026 年 9 月 21 日。逐页对照记录与图像溯源另存于配套源文件的 audit 目录。